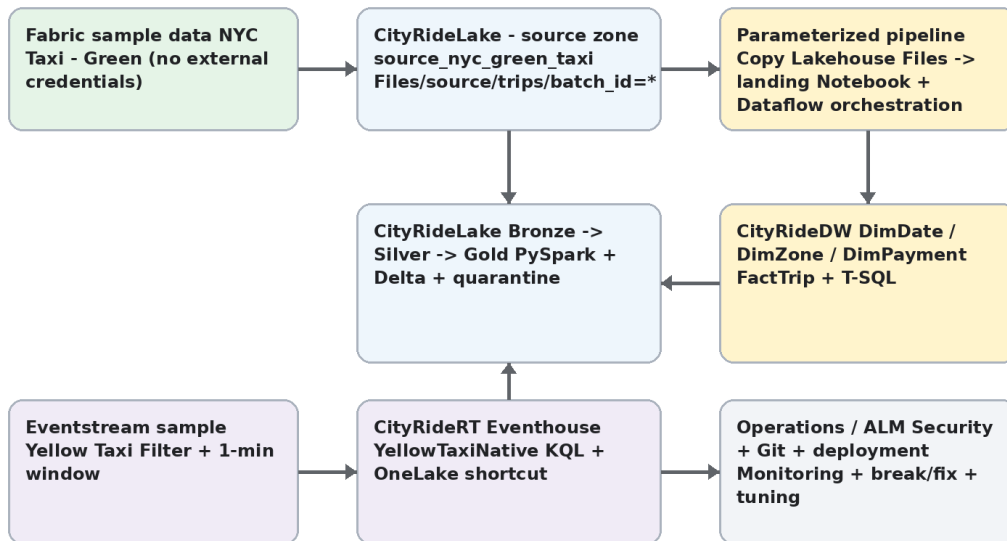


CITYRIDE 360

Complete Microsoft Fabric Data Engineering Capstone

Expanded step-by-step workbook for Exam DP-700

CityRide 360 - Expanded DP-700 architecture



Version 2 - Expanded practical edition | September 2026

Important change in this edition: The batch path no longer depends on anonymous HTTP/CloudFront. It uses Fabric's built-in NYC Taxi - Green sample to seed the Lakehouse, then creates OneLake Parquet batches that the parameterized ingestion pipeline copies and processes. This makes the project reproducible without Azure credentials or an external HTTP connector.

How to use this workbook

This expanded edition keeps the original 15-stage CityRide structure but turns every stage into a build recipe: exact item names, navigation, settings, code, verification queries, failure checks, and a visual showing what the completed stage should resemble.

About the screenshots: The outcome images are intentionally labeled illustrative. I cannot capture your tenant-specific Fabric UI, and Microsoft changes labels frequently. Use each visual as a checkpoint for structure and results, then capture your own real screenshot after you reproduce the outcome.

Track	Recommended use	Estimated effort
Fast exam pass	Stages 1-10, 13-15; read admin-dependent portions of 11-12	14-18 hours
Full capstone	Complete every stage, screenshot checkpoint, and break/fix task	24-35 hours
Portfolio edition	Full capstone plus your own real screenshots, Git history, and a short demo video	30-45 hours

Prerequisites

- A Microsoft Fabric workspace on Fabric capacity or an active Fabric trial.
- Workspace Admin rights are useful. Domain, audit, sensitivity-label, and some OneLake controls can be tenant/admin dependent.
- A GitHub or Azure DevOps repository for Stage 12.
- Optional second test identity or security group to validate restricted access.
- No Azure subscription or external HTTP source is required for the batch ingestion path in this edition.

Updated project naming convention

Item type	Name
Workspace	DP700-CityRide-DEV
Lakehouse	CityRideLake
Warehouse	CityRideDW
Fabric SQL database	CityRideOps
Seed pipeline	pl_cityride_seed
Parameterized pipeline	pl_cityride_ingest
Source-prep notebook	nb_prepare_source
Transform notebook	nb_trip_transform
Dataflow Gen2	df_zones_clean
Streaming notebook	nb_trip_stream
Eventstream	es_cityride_live
Eventhouse	CityRideRT
KQL queryset	qs_cityride_live
Semantic model	CityRideModel
Deployment workspaces	DP700-CityRide-TEST / DP700-CityRide-PROD

Batch data strategy in this edition

The batch source is deliberately Fabric-native:

Layer	Source	Purpose
Seed	Fabric sample data: NYC Taxi - Green	Avoid external connectivity and credentials.
Source files	OneLake Files/source/trips/batch_id=001..003	Create a realistic file-based source that supports parameterized Copy activity.
Landing	OneLake Files/landing/trips/batch_id=...	Destination of the parameterized ingestion pipeline.
Bronze/Silver/Gold	Delta tables in CityRideLake	Medallion processing with PySpark.
Streaming	Fabric Eventstream sample: Yellow Taxi	Real-time path for Eventhouse/KQL/Spark streaming.

Project map

1. Stage 1: Workspace, settings, and engineering decisions
2. Stage 2: Lakehouse and Fabric-native source data
3. Stage 3: Parameterized batch ingestion: full and incremental
4. Stage 4: Dataflow Gen2 for reference data
5. Stage 5: PySpark medallion transformations
6. Stage 6: Dimensional Warehouse and T-SQL
7. Stage 7: Fabric SQL database and mirroring
8. Stage 8: Eventstream, Eventhouse, and KQL
9. Stage 9: Spark Structured Streaming
10. Stage 10: OneLake shortcuts and query acceleration
11. Stage 11: Security and governance
12. Stage 12: Git, database projects, and deployment pipelines
13. Stage 13: Monitoring, alerts, and systematic break/fix
14. Stage 14: Performance optimization lab
15. Stage 15: Final exam simulation and project sign-off

Stage 1 - Workspace, settings, and engineering decisions

Exam focus: Configure Fabric workspace settings; identify admin-dependent settings; choose the right Fabric item for each workload.

Microsoft reference(s): [DP-700 interactive exercises](#) | [Fabric workspace administration](#)

1.1 Create the DEV workspace

In Fabric, open Workspaces > + New workspace. Name it DP700-CityRide-DEV. Add the description "DP-700 CityRide capstone".

Assign it to a Fabric-enabled capacity or trial. If the capacity field is unavailable, confirm with your tenant admin that Fabric workloads are enabled.

Expected result: The workspace opens with an empty item list and the workspace name shown in the header.

Fabric checkpoint - Step 1.1

Expected output

Create the DEV workspace

SUCCESS CHECK

The workspace opens with an empty item list and the workspace name shown in the header.

Illustrative checkpoint - capture your own tenant screenshot after reproducing this result.

Step 1.1 expected outcome (illustrative)

If it does not match: If Fabric item types are unavailable, the workspace is probably not on a Fabric-enabled capacity.

1.2 Inspect Spark and engineering settings

Open Workspace settings. Find the Data Engineering/Data Science or Spark section. Record the default runtime, starter pool/custom pool, autoscale behavior, executor size, and dynamic allocation options you can see.

Do not change compute yet. The exam skill is knowing where to configure it and when a measured bottleneck justifies a change.

Expected result: You have a short settings note containing the current runtime and pool behavior.

Fabric checkpoint - Step 1.2

Expected output

Inspect Spark and engineering settings

SUCCESS CHECK

You have a short settings note containing the current runtime and pool behavior.

Illustrative checkpoint - capture your own tenant screenshot after reproducing this result.

Step 1.2 expected outcome (illustrative)

1.3 Inspect OneLake, domain, and Airflow settings

In Workspace settings, inspect delegated OneLake settings if your tenant exposes them. If the page is absent, record "tenant/admin dependent" rather than treating it as a failed task.

If you have domain rights, assign the workspace to a Transportation domain. Otherwise record the role that would be required.

Inspect Data Factory > Apache Airflow runtime settings and identify Starter Pool versus Custom Pool. Keep Starter Pool for this capstone.

Expected result: Your notes distinguish settings you could configure from settings that require organizational permissions.

Fabric checkpoint - Step 1.3

Expected output

Inspect OneLake, domain, and Airflow settings

SUCCESS CHECK

Your notes distinguish settings you could configure from settings that require organizational permissions.

Illustrative checkpoint - capture your own tenant screenshot after reproducing this result.

Step 1.3 expected outcome (illustrative)

1.4 Create the core Fabric items

From the workspace, select + New item and create: CityRideLake (Lakehouse), CityRideDW (Warehouse), CityRideOps (SQL database), CityRideRT (Eventhouse).

Also create empty notebooks nb_prepare_source, nb_trip_transform, and nb_trip_stream. Create pipelines pl_cityride_seed and pl_cityride_ingest.

Expected result: The workspace item list contains the named Lakehouse, Warehouse, SQL database, Eventhouse, notebooks, and pipelines.

Fabric checkpoint - Step 1.4

Expected output

Create the core Fabric items

SUCCESS CHECK

The workspace item list contains the named Lakehouse, Warehouse, SQL database, Eventhouse, notebooks, and pipelines.

Illustrative checkpoint - capture your own tenant screenshot after reproducing this result.

Step 1.4 expected outcome (illustrative)

Expected outcome visual (illustrative)

Workspace

Lakehouse

Data Factory

Warehouse

Real-Time

Monitoring

Workspace and solution items

Workspace

DP700-CityRide-DEV is on Fabric capacity.
Description and contact are set.

Created items

CityRideLake, CityRideDW, CityRideOps,
CityRideRT, notebooks and pipelines are visible in
the workspace.

Settings reviewed

Spark, OneLake delegated settings, domain assignment and Airflow runtime settings have been
inspected/documented.

Illustrative expected outcome - Fabric UI labels may change

The checkpoint is the item inventory plus a written record of Spark/OneLake/domain/Airflow settings.

DP-700 concepts practiced: workspace settings, Spark configuration, domains, OneLake delegated settings, Airflow settings, solution component selection

Be able to answer aloud:

- Why is a Lakehouse the engineering store, but a Warehouse the relational serving layer?
- Which settings are workspace-level versus tenant/admin-level?

Stage 2 - Lakehouse and Fabric-native source data

Exam focus: Create/manage a Lakehouse; understand Files versus Tables; seed realistic taxi data without HTTP or Azure credentials.

Microsoft reference(s): [Data ingestion options for a lakehouse](#) | [Data Factory sample-data pipeline](#) | [Data Factory end-to-end NYC Taxi tutorial](#)

Why the ingestion method changed: The original guide used direct CloudFront HTTP URLs. This edition uses Fabric's built-in NYC Taxi - Green sample as the seed. Microsoft currently exposes that sample directly in the Copy assistant, so there is no external connection, anonymous HTTP validation, Azure account, or secret to manage.

2.1 Create the Lakehouse folder layout

Open CityRideLake. In the Lakehouse explorer, expand Files. Create these folders: source/trips, landing/trips, quarantine, checkpoints.

Do not manually create Delta folders under Tables. Tables will appear when Spark/Dataflow creates registered Delta tables.

Expected result: Files shows source, landing, quarantine, and checkpoints. Tables is still empty or contains only later-created objects.

Fabric checkpoint - Step 2.1

Expected output

Create the Lakehouse folder layout

SUCCESS CHECK
Files shows source, landing, quarantine, and checkpoints. Tables is still empty or contains only later-created objects.

Illustrative checkpoint - capture your own tenant screenshot after reproducing this result.

Step 2.1 expected outcome (illustrative)

2.2 Seed the taxi source with Fabric sample data

Open pl_cityride_seed. On the pipeline canvas choose Copy data assistant (or Copy data > Use copy assistant).

For Source, choose Sample data > NYC Taxi - Green. Select Next after the preview loads.

For Destination, choose OneLake > Lakehouse > CityRideLake. Choose Tables and name the destination table source_nyc_green_taxi. Use Append for this one-time seed.

Save the activity as Seed_NYC_Green and run it. This can take several minutes depending on capacity.

Expected result: CityRideLake > Tables contains source_nyc_green_taxi and previewing it shows taxi fields such as vendorID and lpepPickupDatetime.

Fabric checkpoint - Step 2.2

Expected output

Seed the taxi source with Fabric sample data

SUCCESS CHECK

CityRideLake > Tables contains source_nyc_green_taxi and previewing it shows taxi fields such as vendorID and lpepPickupDatetime.

Illustrative checkpoint - capture your own tenant screenshot after reproducing this result.

Step 2.2 expected outcome (illustrative)

If it does not match: If Sample data is not visible in your tenant, use the current Data Factory Copy Job tutorial to create the sample table first, then return to this workbook. The rest of the project remains unchanged.

2.3 Verify the actual source schema before coding

```
PYTHON
df = spark.table("source_nyc_green_taxi")
print(df.count())
df.printSchema()
display(df.limit(10))
```

Run the cell in nb_prepare_source. Confirm the current column spelling and case. The sample commonly exposes camelCase fields such as vendorID, lpepPickupDatetime, lpepDropoffDatetime, passengerCount, tripDistance, puLocationId, doLocationId, paymentType, fareAmount, tipAmount, totalAmount.

If one field is named slightly differently in your tenant, change the mapping in the next step once. Do not keep guessing downstream.

Expected result: The notebook displays a schema and 10 rows from source_nyc_green_taxi.

Fabric checkpoint - Step 2.3

Expected output

Verify the actual source schema before coding

SUCCESS CHECK

The notebook displays a schema and 10 rows from source_nyc_green_taxi.

Illustrative checkpoint - capture your own tenant screenshot after reproducing this result.

Step 2.3 expected outcome (illustrative)

2.4 Create three OneLake file batches for the parameterized pipeline

The seed table is convenient, but the DP-700 ingestion exercise should still move files. In nb_prepare_source, standardize a minimal set of columns and create a deterministic batch_id from 001 to 003.

PYTHON

```
from pyspark.sql import functions as F

src = spark.table("source_nyc_green_taxi")

prepared = (src
    .select(
        F.col("vendorID").alias("VendorID"),
        F.col("lpepPickupDatetime").alias("pickup_ts_raw"),
        F.col("lpepDropoffDatetime").alias("dropoff_ts_raw"),
        F.col("passengerCount").alias("passenger_count"),
        F.col("tripDistance").alias("trip_distance"),
        F.col("puLocationId").alias("PULocationID"),
        F.col("doLocationId").alias("DOLocationID"),
        F.col("paymentType").alias("payment_type"),
        F.col("fareAmount").alias("fare_amount"),
        F.col("tipAmount").alias("tip_amount"),
        F.col("totalAmount").alias("total_amount")
    )
    .withColumn(
        "batch_id",
        F.lpad(
            (F.pmod(F.abs(F.hash("VendorID", "pickup_ts_raw", "PULocationID")), F.lit(3)) +
1).cast("string"),
            3, "0"
        )
    )
)

(prepared.write
    .mode("overwrite")
    .partitionBy("batch_id")
    .parquet("Files/source/trips"))

display(prepared.groupBy("batch_id").count().orderBy("batch_id"))
```

Expected result: Files/source/trips contains batch_id=001, batch_id=002, and batch_id=003 folders with Parquet part files.

Fabric checkpoint - Step 2.4

Expected output

Create three OneLake file batches for the parameterized pipeline

SUCCESS CHECK

Files/source/trips contains batch_id=001, batch_id=002, and batch_id=003 folders with Parquet part files.

Illustrative checkpoint - capture your own tenant screenshot after reproducing this result.

Step 2.4 expected outcome (illustrative)

2.5 Create a deliberately dirty zone-reference source for Dataflow Gen2

Use distinct pickup/drop-off location IDs to create a self-contained reference table. We intentionally add duplicate rows and blank values so Stage 4 has meaningful cleaning work.

PYTHON

```
ids = (prepared.select(F.col("PULocationID").alias("LocationID"))
        .union(prepared.select(F.col("DOLocationID").alias("LocationID")))
        .where(F.col("LocationID").isNotNull())
        .distinct())

ref = (ids
        .withColumn("Borough",
            F.when((F.col("LocationID") % 5)==0, "Manhattan")
            .when((F.col("LocationID") % 5)==1, "Brooklyn")
            .when((F.col("LocationID") % 5)==2, "Queens")
            .when((F.col("LocationID") % 5)==3, "Bronx")
            .otherwise("Staten Island"))
        .withColumn("Zone", F.concat(F.lit("CityRide Zone "), F.col("LocationID")))
        .withColumn("service_zone", F.when(F.col("LocationID") % 2 == 0, "Boro
Zone").otherwise("Yellow Zone")))

# inject small data-quality issues
dirty = (ref
        .withColumn("Borough", F.when(F.col("LocationID") % 17 == 0,
F.lit(None)).otherwise(F.col("Borough")))
        .unionByName(ref.limit(5)))

dirty.write.mode("overwrite").format("delta").saveAsTable("source_zone_reference_raw")
display(spark.table("source_zone_reference_raw").limit(20))
```

Expected result: Tables contains source_zone_reference_raw; it includes duplicate LocationID values and some null Borough values by design.

Fabric checkpoint - Step 2.5

Expected output

Create a deliberately dirty zone-reference source for Dataflow Gen2

SUCCESS CHECK

Tables contains source_zone_reference_raw; it includes duplicate LocationID values and some null Borough values by design.

Illustrative checkpoint - capture your own tenant screenshot after reproducing this result.

Step 2.5 expected outcome (illustrative)

2.6 Verify Files versus Tables

In Lakehouse explorer, click Files/source/trips and open one batch folder. These are raw Parquet objects in OneLake.

Then click Tables/source_nyc_green_taxi. This is a registered Delta table that Spark and the SQL analytics endpoint can query.

Write one sentence in your notes: "Files are general OneLake objects; Tables are managed/registered Delta tables; shortcuts reference data without copying it."

Expected result: You can point to one object in Files and one object in Tables and explain why they behave differently.

Fabric checkpoint - Step 2.6

Expected output

Verify Files versus Tables

SUCCESS CHECK

You can point to one object in Files and one object in Tables and explain why they behave differently.

Illustrative checkpoint - capture your own tenant screenshot after reproducing this result.

Step 2.6 expected outcome (illustrative)

Expected outcome visual (illustrative)

Microsoft Fabric - CityRide 360

Stage 2

Workspace

Lakehouse

Data Factory

Warehouse

Real-Time

Monitoring

Seed the batch source without HTTP

Copy data assistant

Source = Sample data > NYC Taxi - Green.
Destination = CityRideLake table
source_nyc_green_taxi.

Lakehouse Files

Files/source/trips contains batch_id=001,
batch_id=002, batch_id=003 Parquet folders.

Reference source

source_zone_reference_raw exists with intentional duplicates/blanks for Dataflow practice.

Illustrative expected outcome - Fabric UI labels may change

The completed stage has a seeded sample table, three source batch folders, and a dirty reference table - all inside Fabric.

DP-700 concepts practiced: Lakehouse, Files vs Tables, Fabric sample data, Parquet, Delta, source preparation

Be able to answer aloud:

- Why seed with sample data, then create file batches instead of querying the sample table directly in every stage?
- What is the difference between a file in OneLake and a registered Lakehouse Delta table?

Stage 3 - Parameterized batch ingestion: full and incremental

Exam focus: Use Pipeline Copy Data, parameters, dynamic expressions, schedule/event triggers, and distinguish full load from incremental file load.

Microsoft reference(s): [Ingest data with a pipeline](#) | [Copy data with Data Factory](#)

3.1 Create pipeline parameters

Open pl_cityride_ingest. On the pipeline canvas, open the Parameters pane and create the following parameters.

Expected result: The pipeline Parameters pane lists the four parameters below.

Fabric checkpoint - Step 3.1

Expected output

Create pipeline parameters

SUCCESS CHECK

The pipeline Parameters pane lists the four parameters below.

Illustrative checkpoint - capture your own tenant screenshot after reproducing this result.

Step 3.1 expected outcome (illustrative)

Parameter	Type	Default	Purpose
p_batch_id	String	001	Select source/trips/batch_id=<value>.
p_full_load	Boolean	true	Overwrite Bronze on the first load; append on later batches.
p_source_root	String	source/trips	Base source folder.
p_target_root	String	landing/trips	Base landing folder.

3.2 Add the parameterized Copy activity

Add a Copy data activity and rename it Copy_Batch_Files.

Source: choose Lakehouse > CityRideLake > Files. Select Wildcard file path if the assistant asks for a file selection mode.

Use the dynamic source folder expression below. The wildcard filename should be *.parquet.

FABRIC EXPRESSION

```
@concat(  
  pipeline().parameters.p_source_root,  
  '/batch_id=',  
  pipeline().parameters.p_batch_id  
)
```

Destination: choose Lakehouse > CityRideLake > Files. Use the dynamic destination folder below.

FABRIC EXPRESSION

```
@concat(  
  pipeline().parameters.p_target_root,  
  '/batch_id=',  
  pipeline().parameters.p_batch_id  
)
```

Use Preserve hierarchy unless your UI exposes a simpler flatten/copy option. The goal is that each batch arrives in its own landing folder.

Expected result: The Copy activity shows Lakehouse Files as source and destination, with dynamic-content expressions rather than hard-coded batch numbers.

Fabric checkpoint - Step 3.2

Expected output

Add the parameterized Copy activity

SUCCESS CHECK

The Copy activity shows Lakehouse Files as source and destination, with dynamic-content expressions rather than hard-coded batch numbers.

Illustrative checkpoint - capture your own tenant screenshot after reproducing this result.

Step 3.2 expected outcome (illustrative)

If it does not match: If the dynamic picker cannot browse a parameterized folder, first configure a static valid folder, save the activity, then replace the folder text with Add dynamic content.

3.3 Add reference refresh only for full load

Add a Dataflow activity after Copy_Batch_Files and point it to df_zones_clean (you build it in Stage 4).

If your pipeline UI supports an If Condition activity, add If_Full_Load with expression `@equals(pipeline().parameters.p_full_load, true)` and place the Dataflow inside the true branch. If you

prefer simplicity while learning, leave the Dataflow in the pipeline and run it on both loads; the destination is Replace mode so the result is idempotent.

Expected result: The pipeline shows either an If Condition branch or a clearly documented unconditional Dataflow refresh.

Fabric checkpoint - Step 3.3

Expected output

Add reference refresh only for full load

SUCCESS CHECK

The pipeline shows either an If Condition branch or a clearly documented unconditional Dataflow refresh.

Illustrative checkpoint - capture your own tenant screenshot after reproducing this result.

Step 3.3 expected outcome (illustrative)

3.4 Add the transformation notebook and map parameters

Add a Notebook activity named Transform_Trips. Select nb_trip_transform.

Under Base parameters, pass `p_batch_id = @pipeline().parameters.p_batch_id` and `p_full_load = @pipeline().parameters.p_full_load`.

Connect dependencies: Copy_Batch_Files -> (Dataflow or If condition) -> Transform_Trips using Success conditions.

Expected result: The notebook activity has two base parameter mappings and sits on the success path after ingestion/reference refresh.

Fabric checkpoint - Step 3.4

Expected output

Add the transformation notebook and map parameters

SUCCESS CHECK

The notebook activity has two base parameter mappings and sits on the success path after ingestion/reference refresh.

Illustrative checkpoint - capture your own tenant screenshot after reproducing this result.

Step 3.4 expected outcome (illustrative)

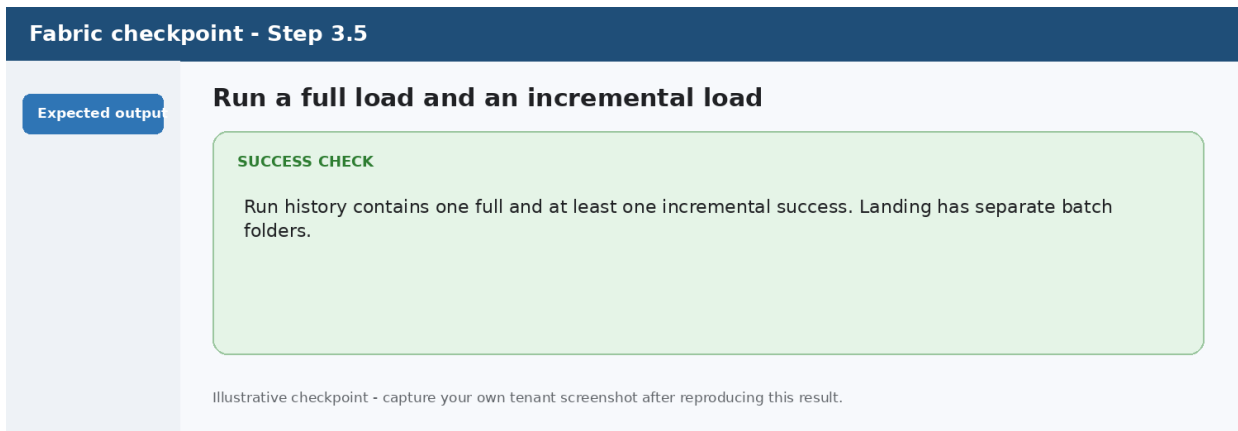
3.5 Run a full load and an incremental load

Run 1: p_batch_id=001, p_full_load=true. After success, verify Files/landing/trips/batch_id=001 and Bronze/Silver/Gold tables after Stage 5 code is in place.

Run 2: p_batch_id=002, p_full_load=false. Verify a new landing folder appears and the notebook appends/merges rather than replacing previous Bronze data.

Optional Run 3: p_batch_id=003, p_full_load=false.

Expected result: Run history contains one full and at least one incremental success. Landing has separate batch folders.



Step 3.5 expected outcome (illustrative)

3.6 Practice schedules and event-trigger reasoning

Create a temporary schedule for the pipeline, such as daily at a time you can observe. Save it, inspect the trigger configuration, then disable it to avoid unnecessary capacity use.

Open the event-trigger options and write when you would choose them: use an event trigger when a new file/event should initiate work instead of waiting for a fixed clock schedule.

Expected result: You can distinguish schedule-driven orchestration from event-driven orchestration even if you keep the trigger disabled.

Fabric checkpoint - Step 3.6

Expected output

Practice schedules and event-trigger reasoning

SUCCESS CHECK

You can distinguish schedule-driven orchestration from event-driven orchestration even if you keep the trigger disabled.

Illustrative checkpoint - capture your own tenant screenshot after reproducing this result.

Step 3.6 expected outcome (illustrative)

3.7 Inject a source-path failure

Run the pipeline with p_batch_id=999. Open the failed Copy_Batch_Files activity and inspect error details, evaluated dynamic path, activity input/output, and run duration.

Restore p_batch_id=001 or 002 and rerun successfully. Save the failure message in your incident notes.

Expected result: You have one deliberate failed run and one successful recovery run in Monitoring/Run history.

Fabric checkpoint - Step 3.7

Expected output

Inject a source-path failure

SUCCESS CHECK

You have one deliberate failed run and one successful recovery run in Monitoring/Run history.

Illustrative checkpoint - capture your own tenant screenshot after reproducing this result.

Step 3.7 expected outcome (illustrative)

Expected outcome visual (illustrative)

Workspace

Lakehouse

Data Factory

Warehouse

Real-Time

Monitoring

Parameterized ingestion pipeline

Parameters

p_batch_id = 001, p_full_load = true/false,
source/target folder values are visible.

Pipeline canvas

CopyBatchFiles -> Dataflow (full load only) ->
nb_trip_transform. Success dependencies are
connected.

Run output

Full run 001 succeeds; incremental run 002 succeeds. Activity output shows files/rows and duration.

Illustrative expected outcome - Fabric UI labels may change

The pipeline checkpoint is a parameterized Lakehouse-to-Lakehouse file copy followed by Dataflow and notebook orchestration.

DP-700 concepts practiced: Copy activity, pipeline parameters, dynamic expressions, full load, incremental file load, schedule triggers, event triggers, troubleshooting

Be able to answer aloud:

- What makes this incremental even though there is no source CDC?
- When would a watermark or CDC pattern be more appropriate than batch folders?

Stage 4 - Dataflow Gen2 for reference data

Exam focus: Use low-code Power Query transformations and explain why Dataflow Gen2 is the right tool for a small reference-cleaning workload.

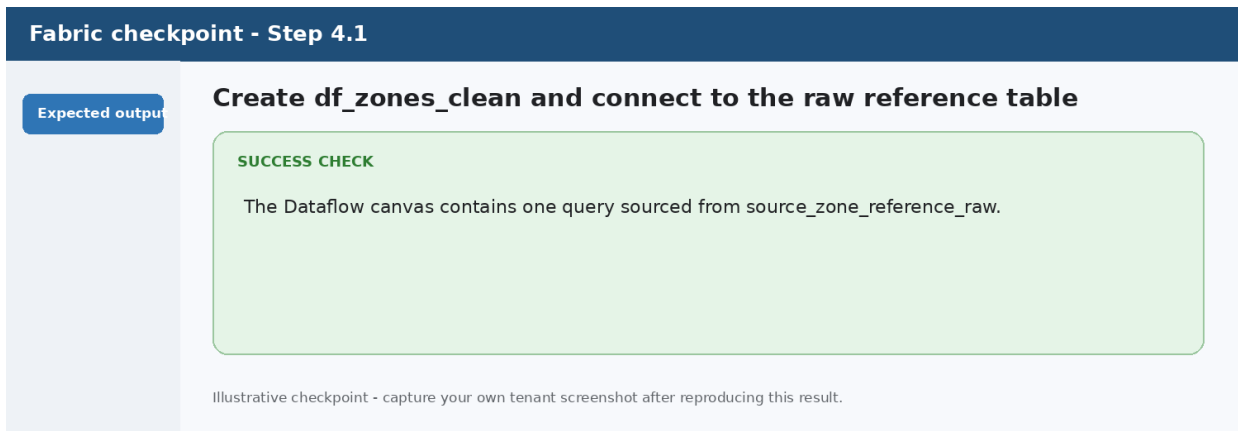
Microsoft reference(s): [Create and use Dataflows Gen2](#) | [Dataflow Gen2 overview](#)

4.1 Create df_zones_clean and connect to the raw reference table

From the workspace, create a Dataflow Gen2 named df_zones_clean.

Choose Get data > Lakehouse (or OneLake catalog), select CityRideLake, and choose source_zone_reference_raw. Confirm the preview includes LocationID, Borough, Zone, service_zone.

Expected result: The Dataflow canvas contains one query sourced from source_zone_reference_raw.



Step 4.1 expected outcome (illustrative)

4.2 Apply the cleaning steps in order

Keep only LocationID, Borough, Zone, service_zone.

Set LocationID to Whole number. Set the three text columns to Text.

Transform > Format > Trim on Borough, Zone, service_zone.

Replace null/blank Borough and Zone with Unknown. If Power Query distinguishes null from empty string, handle both.

Remove duplicates using LocationID as the business key.

Rename service_zone to ServiceZone.

Expected result: Applied Steps shows a short, readable chain: Source -> Choose columns ->

Changed type -> Trimmed text -> Replaced values -> Removed duplicates -> Renamed columns.

Fabric checkpoint - Step 4.2

Expected output

Apply the cleaning steps in order

SUCCESS CHECK
Applied Steps shows a short, readable chain: Source -> Choose columns -> Changed type -> Trimmed text -> Replaced values -> Removed duplicates -> Renamed columns.

Illustrative checkpoint - capture your own tenant screenshot after reproducing this result.

Step 4.2 expected outcome (illustrative)

4.3 Configure the destination table

Choose Add data destination > Lakehouse. Select CityRideLake.

Destination table: dbo.silver_zones. Use Replace/overwrite behavior for this small reference table.

Publish/save the Dataflow and run a refresh.

Expected result: CityRideLake > Tables contains silver_zones with one row per LocationID.

Fabric checkpoint - Step 4.3

Expected output

Configure the destination table

SUCCESS CHECK
CityRideLake > Tables contains silver_zones with one row per LocationID.

Illustrative checkpoint - capture your own tenant screenshot after reproducing this result.

Step 4.3 expected outcome (illustrative)

4.4 Validate the cleaned result

```
SQL
SELECT COUNT(*) AS Rows,
       COUNT(DISTINCT LocationID) AS DistinctLocations
```

```
FROM CityRideLake.dbo.silver_zones;  
  
SELECT TOP 20 *  
FROM CityRideLake.dbo.silver_zones  
WHERE Borough = 'Unknown' OR Zone = 'Unknown';
```

Rows should equal DistinctLocations after duplicate removal. Unknown values may remain because we deliberately injected missing data and replaced them rather than deleting the rows.

Expected result: The validation proves one row per LocationID and shows any intentionally repaired Unknown values.

Fabric checkpoint - Step 4.4

Expected output

Validate the cleaned result

SUCCESS CHECK

The validation proves one row per LocationID and shows any intentionally repaired Unknown values.

Illustrative checkpoint - capture your own tenant screenshot after reproducing this result.

Step 4.4 expected outcome (illustrative)

Expected outcome visual (illustrative)

Workspace

Lakehouse

Data Factory

Warehouse

Real-Time

Monitoring

Dataflow Gen2 - clean reference data

Power Query steps

Source, Keep columns, Type, Trim, Replace nulls, Remove duplicates, Rename columns.

Destination

CityRideLake.dbo.silver_zones in Replace mode.

Result

One row per LocationID with Borough, Zone and ServiceZone populated.

Illustrative expected outcome - Fabric UI labels may change

The Dataflow checkpoint is a clean silver_zones Delta table plus a visible, explainable Applied Steps list.

DP-700 concepts practiced: Dataflow Gen2, Power Query, schema/type cleanup, deduplication, low-code transformation, tool selection

Be able to answer aloud:

- Why use Dataflow Gen2 for this reference table instead of PySpark?
- When would a notebook become the better choice?

Stage 5 - PySpark medallion transformations

Exam focus: Implement parameterized PySpark, Bronze/Silver/Gold Delta tables, quarantine, data quality, denormalization, and aggregation.

Microsoft reference(s): [Analyze data with Apache Spark](#) | [Delta Lake in Fabric](#)

5.1 Create notebook parameter cells

Open nb_trip_transform and attach CityRideLake as the default Lakehouse.

Create the first code cell below and mark it as a parameter cell so Pipeline base parameters can override it.

```
PYTHON
p_batch_id = "001"
p_full_load = True
```

Expected result: The notebook shows a parameter badge/setting on the first cell, and manual execution still works with the defaults.

Fabric checkpoint - Step 5.1

Expected output

Create notebook parameter cells

SUCCESS CHECK

The notebook shows a parameter badge/setting on the first cell, and manual execution still works with the defaults.

Illustrative checkpoint - capture your own tenant screenshot after reproducing this result.

Step 5.1 expected outcome (illustrative)

5.2 Read the landed batch and add ingestion metadata

```
PYTHON
from pyspark.sql import functions as F

source_path = f"Files/landing/trips/batch_id={p_batch_id}/*.parquet"
raw = spark.read.parquet(source_path)

bronze_batch = (raw
    .withColumn("batch_id", F.lit(p_batch_id))
    .withColumn("ingested_at", F.current_timestamp()))

print("batch rows:", bronze_batch.count())
display(bronze_batch.limit(10))
```


Do not transform business values in Bronze beyond metadata. Bronze should preserve source fidelity.

Expected result: A table preview appears and includes batch_id and ingested_at.

Fabric checkpoint - Step 5.2

Expected output

Read the landed batch and add ingestion metadata

SUCCESS CHECK
A table preview appears and includes batch_id and ingested_at.

Illustrative checkpoint - capture your own tenant screenshot after reproducing this result.

Step 5.2 expected outcome (illustrative)

5.3 Write Bronze with full/incremental semantics

PYTHON

```
mode = "overwrite" if p_full_load else "append"

(bronze_batch.write
 .format("delta")
 .mode(mode)
 .option("overwriteSchema", "true")
 .saveAsTable("dbo.bronze_trips"))

print("bronze total:", spark.table("dbo.bronze_trips").count())
```

For the full run, Bronze is replaced by batch 001. For incremental runs, new batches append. Because the batch file source is deterministic, do not rerun the same incremental batch repeatedly unless you also implement a merge/idempotency key.

Expected result: dbo.bronze_trips exists. Its row count increases after a new incremental batch.

Fabric checkpoint - Step 5.3

Expected output

Write Bronze with full/incremental semantics

SUCCESS CHECK

dbo.bronze_trips exists. Its row count increases after a new incremental batch.

Illustrative checkpoint - capture your own tenant screenshot after reproducing this result.

Step 5.3 expected outcome (illustrative)

5.4 Clean and standardize Silver

PYTHON

```
b = spark.table("dbo.bronze_trips")

silver = (b
    .withColumn("pickup_ts", F.to_timestamp("pickup_ts_raw"))
    .withColumn("dropoff_ts", F.to_timestamp("dropoff_ts_raw"))
    .withColumn("pickup_date", F.to_date("pickup_ts_raw"))
    .withColumn("trip_duration_min",
        (F.unix_timestamp("dropoff_ts_raw") - F.unix_timestamp("pickup_ts_raw")) / 60.0)
    .withColumn("passenger_count", F.coalesce(F.col("passenger_count"), F.lit(1)))
    .filter(F.col("pickup_ts").isNotNull())
    .filter(F.col("dropoff_ts").isNotNull())
    .filter(F.col("trip_distance") >= 0)
    .filter(F.col("total_amount") >= 0)
    .filter(F.col("trip_duration_min").between(0, 24*60))
    .dropDuplicates([
        "VendorID", "pickup_ts_raw", "dropoff_ts_raw",
        "PULocationID", "DOLocationID", "total_amount"
    ]))

(silver.write.format("delta").mode("overwrite").saveAsTable("dbo.silver_trips"))
```

Expected result: silver_trips exists and contains parsed timestamps, pickup_date, and trip_duration_min with invalid/duplicate rows removed.

Fabric checkpoint - Step 5.4

Expected output

Clean and standardize Silver

SUCCESS CHECK

silver_trips exists and contains parsed timestamps, pickup_date, and trip_duration_min with invalid/duplicate rows removed.

Illustrative checkpoint - capture your own tenant screenshot after reproducing this result.

Step 5.4 expected outcome (illustrative)

5.5 Quarantine invalid rows

PYTHON

```
invalid = (b
    .filter(
        F.col("pickup_ts_raw").isNull() |
        F.col("dropoff_ts_raw").isNull() |
        (F.col("trip_distance") < 0) |
        (F.col("total_amount") < 0)
    ))

(invalid.write
    .mode("overwrite")
    .parquet("Files/quarantine/trips"))

print("quarantined:", invalid.count())
```

Expected result: Files/quarantine/trips contains Parquet files. The count may be zero if the sample happens to be clean; the logic itself is still valid.

Fabric checkpoint - Step 5.5

Expected output

Quarantine invalid rows

SUCCESS CHECK

Files/quarantine/trips contains Parquet files. The count may be zero if the sample happens to be clean; the logic itself is still valid.

Illustrative checkpoint - capture your own tenant screenshot after reproducing this result.

Step 5.5 expected outcome (illustrative)

5.6 Join zones and create Gold metrics

PYTHON

```
trips = spark.table("dbo.silver_trips")
zones = spark.table("dbo.silver_zones")

pu = zones.select(
    F.col("LocationID").alias("PULocationID"),
    F.col("Borough").alias("PickupBorough"),
    F.col("Zone").alias("PickupZone"))

do = zones.select(
    F.col("LocationID").alias("DOLocationID"),
    F.col("Borough").alias("DropoffBorough"),
    F.col("Zone").alias("DropoffZone"))

enriched = (trips
    .join(F.broadcast(pu), "PULocationID", "left")
    .join(F.broadcast(do), "DOLocationID", "left")
    .fillna({
        "PickupBorough": "Unknown", "PickupZone": "Unknown",
        "DropoffBorough": "Unknown", "DropoffZone": "Unknown"
    }))

gold = (enriched
    .groupBy("pickup_date", "PULocationID", "PickupBorough", "PickupZone")
    .agg(
        F.count("*").alias("TripCount"),
        F.round(F.sum("total_amount"), 2).alias("Revenue"),
        F.round(F.avg("fare_amount"), 2).alias("AvgFare"),
        F.round(F.avg("trip_distance"), 2).alias("AvgDistance"),
        F.round(F.avg("trip_duration_min"), 2).alias("AvgDurationMin"),
        F.sum("passenger_count").alias("Passengers")
    ))

gold.write.format("delta").mode("overwrite").saveAsTable("dbo.gold_daily_zone_metrics")

payment = (enriched.groupBy("pickup_date", "payment_type")
    .agg(F.count("*").alias("TripCount"), F.sum("total_amount").alias("Revenue")))
payment.write.format("delta").mode("overwrite").saveAsTable("dbo.gold_payment_metrics")
```

Expected result: Gold tables exist and contain aggregated daily/zone and payment metrics.

Fabric checkpoint - Step 5.6

Expected output

Join zones and create Gold metrics

SUCCESS CHECK

Gold tables exist and contain aggregated daily/zone and payment metrics.

Illustrative checkpoint - capture your own tenant screenshot after reproducing this result.

5.7 Run explicit quality checks

PYTHON

```
bronze_count = spark.table("dbo.bronze_trips").count()
silver_count = spark.table("dbo.silver_trips").count()
print("bronze", bronze_count, "silver", silver_count)

assert spark.table("dbo.silver_trips").filter("total_amount < 0").count() == 0
assert spark.table("dbo.silver_trips").filter("trip_distance < 0").count() == 0

(display(spark.table("dbo.gold_daily_zone_metrics")
    .filter("PickupZone = 'Unknown'")
    .groupBy("PULocationID").count()
    .orderBy(F.desc("count"))
    .limit(20)))
```

Expected result: The assertions pass. Bronze row count is greater than or equal to Silver. Unknown zone IDs are visible if the reference table does not cover every ID.

Fabric checkpoint - Step 5.7

Expected output

Run explicit quality checks

SUCCESS CHECK

The assertions pass. Bronze row count is greater than or equal to Silver. Unknown zone IDs are visible if the reference table does not cover every ID.

Illustrative checkpoint - capture your own tenant screenshot after reproducing this result.

Expected outcome visual (illustrative)

Workspace

Lakehouse

Data Factory

Warehouse

Real-Time

Monitoring

PySpark medallion transformation

Notebook parameters

p_batch_id and p_full_load are marked as parameters.

Lakehouse tables

bronze_trips, silver_trips, gold_daily_zone_metrics and gold_payment_metrics are visible under Tables.

Quality checks

No negative total_amount in Silver; invalid rows exist in Files/quarantine; unknown zones are counted.

Illustrative expected outcome - Fabric UI labels may change

The notebook checkpoint is a visible Bronze/Silver/Gold table set plus repeatable quality checks and quarantine logic.

DP-700 concepts practiced: PySpark, Delta tables, medallion architecture, duplicates, missing/invalid data, broadcast join, denormalization, aggregation, parameters

Be able to answer aloud:

- What belongs in Bronze versus Silver versus Gold?
- Why is the zone reference broadcastable?
- How would you make incremental re-runs idempotent?

Stage 6 - Dimensional Warehouse and T-SQL

Exam focus: Prepare/load a star schema; use T-SQL joins, CTEs, aggregations, and window functions; compare Warehouse with Lakehouse serving.

Microsoft reference(s): [Data warehouse labs](#) | [Fabric Data Warehouse overview](#)

6.1 Create the star-schema tables

Open CityRideDW > New SQL query. Run the DDL below. Fabric Warehouse does not require you to create traditional clustered indexes for this exercise; focus on the logical dimensional model.

SQL

```
CREATE TABLE dbo.DimDate (
    DateKey int NOT NULL, FullDate date NOT NULL,
    CalendarYear smallint NOT NULL, CalendarMonth tinyint NOT NULL,
    MonthName varchar(12) NOT NULL, DayOfWeek tinyint NOT NULL, DayName varchar(12) NOT NULL
);

CREATE TABLE dbo.DimZone (
    ZoneKey int NOT NULL, Borough varchar(100), ZoneName varchar(150), ServiceZone varchar(100)
);

CREATE TABLE dbo.DimPayment (PaymentKey int NOT NULL, PaymentName varchar(50) NOT NULL);

CREATE TABLE dbo.FactTrip (
    PickupDateKey int NOT NULL, PickupZoneKey int, DropoffZoneKey int, PaymentKey int,
    PassengerCount int, TripDistance decimal(12,2), TripDurationMin decimal(12,2),
    FareAmount decimal(12,2), TipAmount decimal(12,2), TotalAmount decimal(12,2)
);
```

Expected result: The Warehouse explorer shows DimDate, DimZone, DimPayment, and FactTrip.

Fabric checkpoint - Step 6.1

Expected output

Create the star-schema tables

SUCCESS CHECK

The Warehouse explorer shows DimDate, DimZone, DimPayment, and FactTrip.

Illustrative checkpoint - capture your own tenant screenshot after reproducing this result.

Step 6.1 expected outcome (illustrative)

6.2 Load the dimensions from curated Lakehouse data

```
SQL
INSERT INTO dbo.DimPayment VALUES
(0,'Unknown'),(1,'Credit card'),(2,'Cash'),(3,'No charge'),(4,'Dispute'),(5,'Unknown/other'),
(6,'Voided trip');

INSERT INTO dbo.DimZone (ZoneKey, Borough, ZoneName, ServiceZone)
SELECT DISTINCT LocationID, Borough, Zone, ServiceZone
FROM CityRideLake.dbo.silver_zones;

INSERT INTO dbo.DimDate
SELECT DISTINCT
YEAR(pickup_date)*10000 + MONTH(pickup_date)*100 + DAY(pickup_date),
pickup_date, YEAR(pickup_date), MONTH(pickup_date), DATENAME(month,pickup_date),
DATEPART(weekday,pickup_date), DATENAME(weekday,pickup_date)
FROM CityRideLake.dbo.silver_trips;
```

If cross-item three-part names do not autocomplete immediately, refresh the workspace/SQL endpoint metadata and verify the Lakehouse SQL analytics endpoint is online.

Expected result: Each dimension contains rows and DimDate covers the pickup dates in silver_trips.

Fabric checkpoint - Step 6.2

Expected output

Load the dimensions from curated Lakehouse data

SUCCESS CHECK

Each dimension contains rows and DimDate covers the pickup dates in silver_trips.

Illustrative checkpoint - capture your own tenant screenshot after reproducing this result.

Step 6.2 expected outcome (illustrative)

6.3 Load the fact table

```
SQL
TRUNCATE TABLE dbo.FactTrip;

INSERT INTO dbo.FactTrip
SELECT
YEAR(s.pickup_date)*10000 + MONTH(s.pickup_date)*100 + DAY(s.pickup_date),
s.PULocationID, s.DOLocationID, COALESCE(s.payment_type,0),
CAST(s.passenger_count AS int), CAST(s.trip_distance AS decimal(12,2)),
CAST(s.trip_duration_min AS decimal(12,2)), CAST(s.fare_amount AS decimal(12,2)),
CAST(s.tip_amount AS decimal(12,2)), CAST(s.total_amount AS decimal(12,2))
FROM CityRideLake.dbo.silver_trips AS s;
```

Expected result: FactTrip contains trip-level analytical rows and foreign-key-style columns map

to the dimensions.

Fabric checkpoint - Step 6.3

Expected output

Load the fact table

SUCCESS CHECK

FactTrip contains trip-level analytical rows and foreign-key-style columns map to the dimensions.

Illustrative checkpoint - capture your own tenant screenshot after reproducing this result.

Step 6.3 expected outcome (illustrative)

6.4 Run analytical and consistency queries

```
SQL
WITH DailyRevenue AS (
    SELECT PickupDateKey, PickupZoneKey, COUNT(*) AS TripCount, SUM(TotalAmount) AS Revenue
    FROM dbo.FactTrip
    GROUP BY PickupDateKey, PickupZoneKey
)
SELECT PickupDateKey, PickupZoneKey, TripCount, Revenue,
       SUM(Revenue) OVER (PARTITION BY PickupZoneKey ORDER BY PickupDateKey ROWS UNBOUNDED
       PRECEDING) AS RunningRevenue
FROM DailyRevenue
ORDER BY PickupZoneKey, PickupDateKey;

SELECT COUNT(*) AS NegativeAmounts
FROM dbo.FactTrip
WHERE TotalAmount < 0 OR FareAmount < 0;
```

Expected result: The running-revenue query returns ordered rows and NegativeAmounts is zero.

Fabric checkpoint - Step 6.4

Expected output

Run analytical and consistency queries

SUCCESS CHECK

The running-revenue query returns ordered rows and NegativeAmounts is zero.

Illustrative checkpoint - capture your own tenant screenshot after reproducing this result.

Step 6.4 expected outcome (illustrative)

Expected outcome visual (illustrative)

Microsoft Fabric - CityRide 360Stage 6

Workspace
Lakehouse
Data Factory
Warehouse
Real-Time
Monitoring

Dimensional Warehouse

Tables
DimDate, DimZone, DimPayment and FactTrip exist in CityRideDW.

Star schema
FactTrip joins to date, pickup/dropoff zone and payment dimensions.

Query result
RunningRevenue window query returns ordered results by zone/date.

Illustrative expected outcome - Fabric UI labels may change

The Warehouse checkpoint is a populated star schema plus at least one window-function query and data-quality query.

DP-700 concepts practiced: Warehouse, star schema, T-SQL, CTEs, window functions, cross-item query, dimensional modeling

Be able to answer aloud:

- Why keep a Warehouse if the Lakehouse already has Gold tables?
- When would Lakehouse SQL analytics endpoint be enough?

Stage 7 - Fabric SQL database and mirroring

Exam focus: Implement operational SQL tables, observe automatic mirroring, and distinguish mirroring from Copy Data.

Microsoft reference(s): [SQL Database in Fabric](#) | [Mirroring overview](#)

7.1 Create operational Driver and Vehicle tables

Open CityRideOps and run the following in its SQL editor.

```
SQL
CREATE TABLE dbo.Driver (
    DriverId int PRIMARY KEY, FullName varchar(100) NOT NULL, Email varchar(200),
    LicenseNumber varchar(50), Active bit NOT NULL, ModifiedDate datetime2 NOT NULL DEFAULT
sysdatetime()
);

CREATE TABLE dbo.Vehicle (
    VehicleId int PRIMARY KEY, RegistrationNumber varchar(30) NOT NULL,
    VehicleType varchar(30), ModifiedDate datetime2 NOT NULL DEFAULT sysdatetime()
);

INSERT dbo.Driver (DriverId,FullName,Email,LicenseNumber,Active) VALUES
(1,'Amina Diallo','amina@example.test','LIC-1001',1),
(2,'Marco Silva','marco@example.test','LIC-1002',1),
(3,'Lena Martin','lena@example.test','LIC-1003',0);

INSERT dbo.Vehicle (VehicleId,RegistrationNumber,VehicleType) VALUES
(101,'CITY-101','Sedan'),(102,'CITY-102','EV'),(103,'CITY-103','Van');
```

Expected result: Queries against Driver and Vehicle return the inserted rows.

Fabric checkpoint - Step 7.1

Expected output

Create operational Driver and Vehicle tables

SUCCESS CHECK

Queries against Driver and Vehicle return the inserted rows.

Illustrative checkpoint - capture your own tenant screenshot after reproducing this result.

Step 7.1 expected outcome (illustrative)

7.2 Locate the mirrored analytical representation

Open the SQL database item and its SQL analytics/OneLake analytical surface. The exact label can change as Fabric evolves. Look for the automatically mirrored representation of supported Fabric SQL database tables.

Verify Driver and Vehicle are queryable from the analytical side without creating a Copy pipeline.

Expected result: The same operational tables are visible in the analytical/mirrored surface.

Fabric checkpoint - Step 7.2

Expected output

Locate the mirrored analytical representation

SUCCESS CHECK

The same operational tables are visible in the analytical/mirrored surface.

Illustrative checkpoint - capture your own tenant screenshot after reproducing this result.

Step 7.2 expected outcome (illustrative)

7.3 Prove change propagation

```
SQL
UPDATE dbo.Driver
SET Active = 1, ModifiedDate = sysdatetime()
WHERE DriverId = 3;
```

Wait for replication to catch up, then query the analytical representation and confirm the new value appears.

Expected result: DriverId 3 is Active=1 on both operational and mirrored analytical sides after replication.

Fabric checkpoint - Step 7.3

Expected output

Prove change propagation

SUCCESS CHECK

DriverId 3 is Active=1 on both operational and mirrored analytical sides after replication.

Illustrative checkpoint - capture your own tenant screenshot after reproducing this result.

Step 7.3 expected outcome (illustrative)

7.4 Write the mirroring versus Copy comparison

In your project notes, record: Mirroring continuously replicates supported source changes into an analytical representation with minimal ETL plumbing; Copy Data is an explicit movement/orchestration action that you schedule or trigger.

Expected result: You can explain which pattern fits continuous source replication and which fits controlled batch movement.

Fabric checkpoint - Step 7.4

Expected output

Write the mirroring versus Copy comparison

SUCCESS CHECK

You can explain which pattern fits continuous source replication and which fits controlled batch movement.

Illustrative checkpoint - capture your own tenant screenshot after reproducing this result.

Step 7.4 expected outcome (illustrative)

Expected outcome visual (illustrative)

Workspace

Lakehouse

Data Factory

Warehouse

Real-Time

Monitoring

Fabric SQL database and mirroring

Operational DB

CityRideOps contains Driver and Vehicle tables with sample rows.

Mirrored analytics

The SQL analytics / OneLake representation shows supported mirrored tables.

Change validation

An updated driver value becomes visible after mirroring catches up.

Illustrative expected outcome - Fabric UI labels may change

The checkpoint is a small operational database plus evidence that Fabric mirroring exposes the tables analytically.

DP-700 concepts practiced: Fabric SQL database, mirroring, operational vs analytical storage, Copy Data comparison

Be able to answer aloud:

- When would mirroring be preferable to a scheduled copy?
- Why would you still use pipelines even if mirroring exists?

Stage 8 - Eventstream, Eventhouse, and KQL

Exam focus: Create a real-time path with Fabric sample data; transform events; ingest to Eventhouse; query with KQL and time buckets.

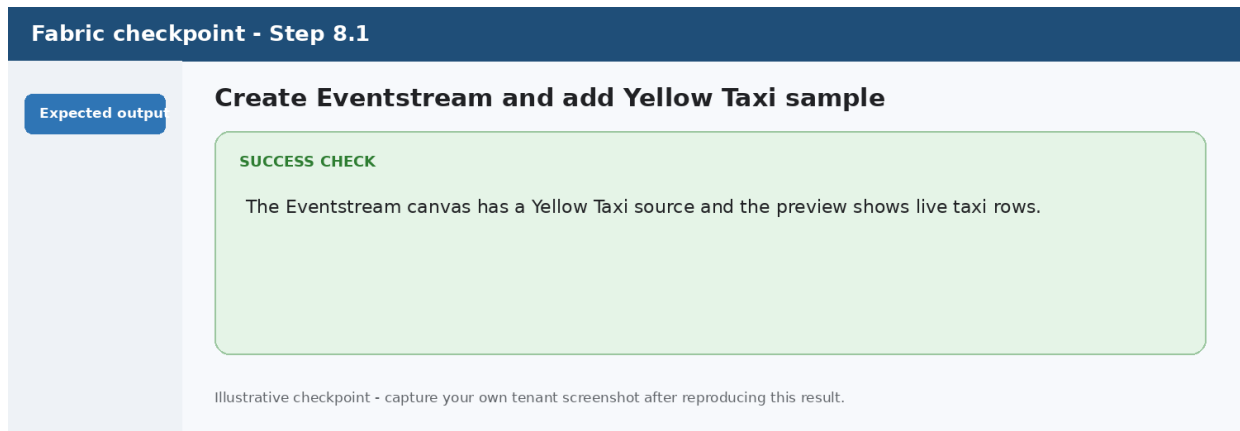
Microsoft reference(s): [Eventstream overview](#) | [Preview Eventstream data](#)

8.1 Create Eventstream and add Yellow Taxi sample

Create or open es_cityride_live. Select Add source > Sample data > Yellow Taxi.

Select the source node and open Data preview. Confirm events are flowing before adding transformations.

Expected result: The Eventstream canvas has a Yellow Taxi source and the preview shows live taxi rows.



Step 8.1 expected outcome (illustrative)

8.2 Add a filter and one-minute aggregation

Add a Filter transformation and keep events where total_amount > 0.

Add a Group by transformation. Use a 1-minute tumbling window. Calculate count of events and, if available in the visual designer, sum(total_amount).

Name the derived stream Taxi_1Min_Aggregates.

Expected result: The canvas shows source -> filter -> group-by/derived stream with data preview at each node.

Fabric checkpoint - Step 8.2

Expected output

Add a filter and one-minute aggregation

SUCCESS CHECK

The canvas shows source -> filter -> group-by/derived stream with data preview at each node.

Illustrative checkpoint - capture your own tenant screenshot after reproducing this result.

Step 8.2 expected outcome (illustrative)

8.3 Create the Eventhouse destination

Add Destination > Eventhouse. Choose CityRideRT and its KQL database. Create a destination table YellowTaxiNative.

Map incoming fields automatically first. Review timestamp/numeric types. Publish the Eventstream and wait for ingestion.

Expected result: CityRideRT contains YellowTaxiNative and recent rows appear in its preview/query.

Fabric checkpoint - Step 8.3

Expected output

Create the Eventhouse destination

SUCCESS CHECK

CityRideRT contains YellowTaxiNative and recent rows appear in its preview/query.

Illustrative checkpoint - capture your own tenant screenshot after reproducing this result.

Step 8.3 expected outcome (illustrative)

8.4 Create the KQL queryset and run three query patterns

Create qs_cityride_live and run the following one at a time.

```
KQL
YellowTaxiNative
| where ingestion_time() > ago(10m)
| project tpep_pickup_datetime, tpep_dropoff_datetime, passenger_count, trip_distance,
fare_amount, tip_amount, total_amount
| take 100
```


KQL

YellowTaxiNative

```
| extend pickup_ts = todatetime(tpep_pickup_datetime)
| summarize Trips=count(), Revenue=sum(todouble(total_amount)),
AvgFare=avg(todouble(fare_amount))
  by bin(pickup_ts, 1m)
| order by pickup_ts desc
```

KQL

YellowTaxiNative

```
| extend pickup_ts=todatetime(tpep_pickup_datetime), dropoff_ts=todatetime(tpep_dropoff_datetime)
| extend duration_min = datetime_diff('minute', dropoff_ts, pickup_ts)
| where duration_min < 0 or todouble(total_amount) < 0
| project pickup_ts, dropoff_ts, duration_min, total_amount
| take 50
```

Expected result: The first query returns recent rows; the second returns 1-minute buckets; the third is an anomaly check that may return zero or a few rows.

Fabric checkpoint - Step 8.4

Expected output

Create the KQL queryset and run three query patterns

SUCCESS CHECK

The first query returns recent rows; the second returns 1-minute buckets; the third is an anomaly check that may return zero or a few rows.

Illustrative checkpoint - capture your own tenant screenshot after reproducing this result.

Step 8.4 expected outcome (illustrative)

Expected outcome visual (illustrative)

Workspace

Lakehouse

Data Factory

Warehouse

Real-Time

Monitoring

Eventstream -> Eventhouse -> KQL**Eventstream canvas**

Yellow Taxi sample -> Filter total_amount > 0 ->
1-minute Group by -> Eventhouse destination.

Eventhouse table

YellowTaxiNative is receiving events. Data preview
contains recent rows.

KQL queryset

1-minute Trips / Revenue / AvgFare query returns time buckets.

Illustrative expected outcome - Fabric UI labels may change

The real-time checkpoint is a published Eventstream feeding a native Eventhouse table plus a working KQL one-minute aggregation.

DP-700 concepts practiced: Eventstream, sample sources, filter/group-by transforms, Eventhouse, KQL, bin(), real-time ingestion

Be able to answer aloud:

- Why use Eventhouse/KQL instead of a Warehouse for this live event workload?
- What work is appropriate in Eventstream before ingestion?

Stage 9 - Spark Structured Streaming

Exam focus: Read Eventstream data from a notebook, use event-time watermark/window logic, and write aggregates to Delta.

Microsoft reference(s): [Eventstream with Spark](#) | [Structured Streaming concepts](#)

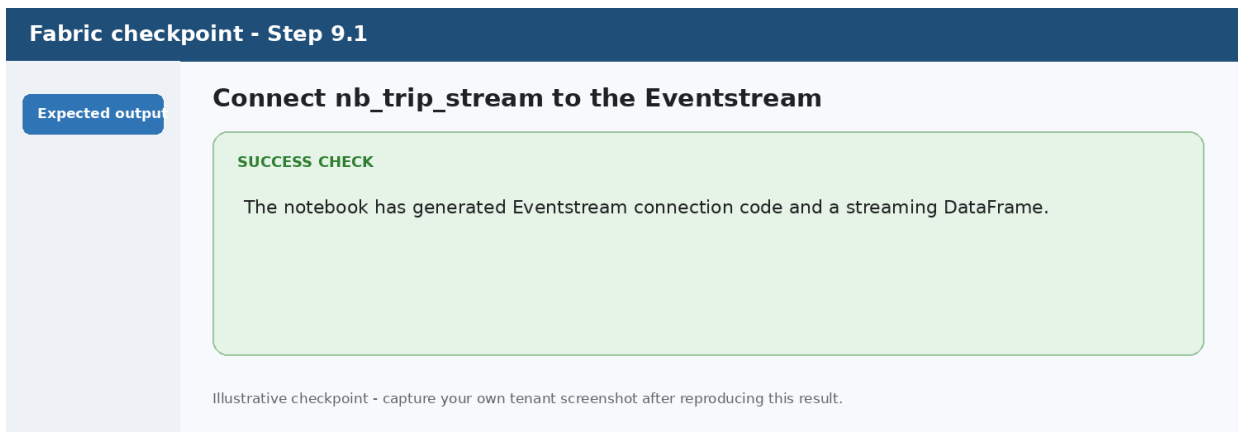
9.1 Connect nb_trip_stream to the Eventstream

Open nb_trip_stream and attach CityRideLake.

In notebook Explorer choose Add data items > From Real-Time hub (wording can vary), find es_cityride_live, and choose Read with Spark. Let Fabric generate the connection/bootstrap code.

Run only the generated read cell first and inspect the streaming schema/preview.

Expected result: The notebook has generated Eventstream connection code and a streaming DataFrame.



Step 9.1 expected outcome (illustrative)

9.2 Parse the payload if Fabric gives you raw value bytes

If your generated DataFrame already has taxi columns, continue. If it has a raw value/binary JSON column, inspect one event and use from_json with the observed schema before aggregating.

Do not paste hard-coded Kafka/Event Hubs connection secrets from someone else's tutorial; use the code generated for your Eventstream.

Expected result: You can call printSchema() and see pickup timestamp, location, distance, and amount fields in a parsed streaming DataFrame.

Fabric checkpoint - Step 9.2

Expected output

Parse the payload if Fabric gives you raw value bytes

SUCCESS CHECK

You can call `printSchema()` and see pickup timestamp, location, distance, and amount fields in a parsed streaming DataFrame.

Illustrative checkpoint - capture your own tenant screenshot after reproducing this result.

Step 9.2 expected outcome (illustrative)

9.3 Apply watermark and five-minute window

PYTHON

```
from pyspark.sql import functions as F

stream_clean = (events
    .withColumn("pickup_ts", F.to_timestamp("tpep_pickup_datetime"))
    .withColumn("total_amount_d", F.col("total_amount").cast("double"))
    .withColumn("trip_distance_d", F.col("trip_distance").cast("double"))
    .filter(F.col("pickup_ts").isNotNull())
    .filter(F.col("total_amount_d") >= 0))

windowed = (stream_clean
    .withWatermark("pickup_ts", "10 minutes")
    .groupBy(F.window("pickup_ts", "5 minutes"), F.col("PULocationID"))
    .agg(F.count("*").alias("TripCount"),
        F.sum("total_amount_d").alias("Revenue"),
        F.avg("trip_distance_d").alias("AvgDistance")))
```

Expected result: The streaming logical plan includes a watermark and windowed aggregation.

Fabric checkpoint - Step 9.3

Expected output

Apply watermark and five-minute window

SUCCESS CHECK

The streaming logical plan includes a watermark and windowed aggregation.

Illustrative checkpoint - capture your own tenant screenshot after reproducing this result.

Step 9.3 expected outcome (illustrative)

9.4 Write the streaming result to Delta with a checkpoint

PYTHON

```
query = (windowed.writeStream
    .format("delta")
    .outputMode("append")
    .option("checkpointLocation", "Files/checkpoints/cityride_trip_windows")
    .toTable("dbo.stream_trip_windows"))
```

Leave it running long enough to produce data, then stop it deliberately from the notebook if you do not want continuous capacity usage.

Expected result: Tables contains stream_trip_windows and Files/checkpoints/cityride_trip_windows contains checkpoint state.

Fabric checkpoint - Step 9.4

Expected output

Write the streaming result to Delta with a checkpoint

SUCCESS CHECK

Tables contains stream_trip_windows and Files/checkpoints/cityride_trip_windows contains checkpoint state.

Illustrative checkpoint - capture your own tenant screenshot after reproducing this result.

Step 9.4 expected outcome (illustrative)

9.5 Explain late data

Write a note: event time is when the taxi event occurred; ingestion time is when Fabric received it. A 10-minute watermark tells Spark how long stateful operators should wait for late events. Very-late events can be excluded from finalized window results.

Longer watermarks improve tolerance for lateness but retain more state and can increase memory/state-store cost.

Expected result: You can explain the business/cost trade-off of the watermark rather than treating 10 minutes as a magic number.

Fabric checkpoint - Step 9.5

Expected output

Explain late data

SUCCESS CHECK

You can explain the business/cost trade-off of the watermark rather than treating 10 minutes as a magic number.

Illustrative checkpoint - capture your own tenant screenshot after reproducing this result.

Step 9.5 expected outcome (illustrative)

Expected outcome visual (illustrative)

Microsoft Fabric - CityRide 360Stage 9

Workspace

Lakehouse

Data Factory

Warehouse

Real-Time

Monitoring

Spark Structured Streaming

Notebook source

nb_trip_stream is connected to es_cityride_live via Real-Time hub generated code.

Watermark + window

10-minute watermark and 5-minute event-time window are applied.

Delta sink

dbo.stream_trip_windows receives windowed aggregates and checkpoint files exist.

Illustrative expected outcome - Fabric UI labels may change

The Structured Streaming checkpoint is a running/stopped query with a Delta sink and checkpoint directory, not just a static Spark DataFrame.

DP-700 concepts practiced: Structured Streaming, event time, watermarks, windows, late data, checkpointing, Delta streaming sink

Be able to answer aloud:

- What problem does a watermark solve?
- Why is checkpointing required for a recoverable streaming query?

Stage 10 - OneLake shortcuts and query acceleration in Real-Time Intelligence

Exam focus: Create a OneLake shortcut from Eventhouse to Lakehouse Gold data; compare standard external query with accelerated shortcut behavior.

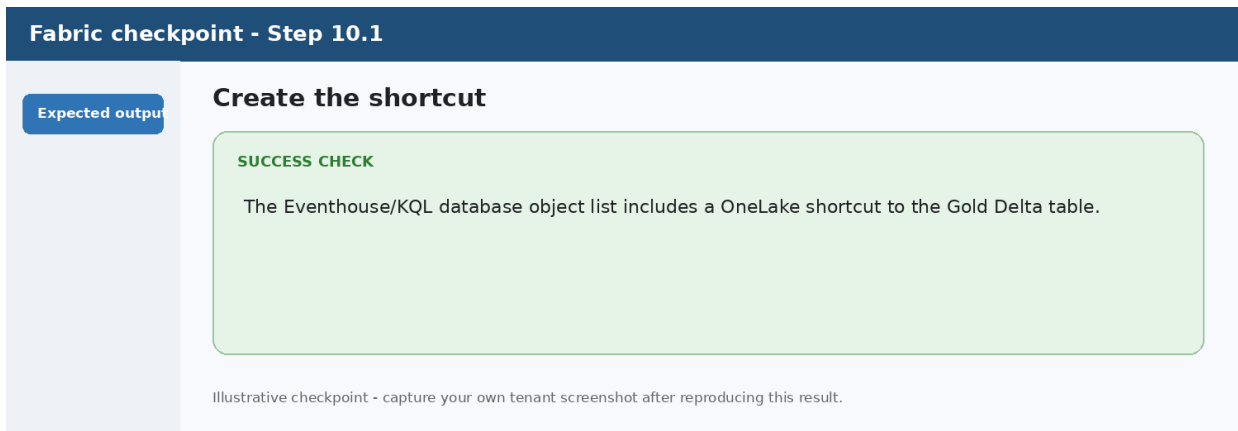
Microsoft reference(s): [OneLake shortcuts](#) | [Query acceleration overview](#)

10.1 Create the shortcut

In CityRideRT KQL database, choose New > OneLake shortcut. Select Microsoft OneLake and browse to CityRideLake > Tables > gold_daily_zone_metrics.

Name the shortcut gold_daily_zone_metrics. Create it with query acceleration OFF initially.

Expected result: The Eventhouse/KQL database object list includes a OneLake shortcut to the Gold Delta table.



Step 10.1 expected outcome (illustrative)

10.2 Run a standard external-table query

```
KQL
external_table('gold_daily_zone_metrics')
| summarize Revenue=sum(todouble(Revenue)), Trips=sum(tolong(TripCount)) by PickupBorough
| order by Revenue desc
```

Record query duration/diagnostic information exposed by your UI. On a small dataset, the difference may be too small to be meaningful.

Expected result: The query returns Gold data without copying it into a native Eventhouse table.

Fabric checkpoint - Step 10.2

Expected output

Run a standard external-table query

SUCCESS CHECK

The query returns Gold data without copying it into a native Eventhouse table.

Illustrative checkpoint - capture your own tenant screenshot after reproducing this result.

Step 10.2 expected outcome (illustrative)

10.3 Enable and test query acceleration

Open shortcut management/data policies and enable query acceleration if your capacity/tenant exposes it. Set a caching policy appropriate for the test.

Run the same query several times and record observations rather than claiming a guaranteed speed-up.

Expected result: Your notes contain the same query tested in standard and accelerated shortcut modes.

Fabric checkpoint - Step 10.3

Expected output

Enable and test query acceleration

SUCCESS CHECK

Your notes contain the same query tested in standard and accelerated shortcut modes.

Illustrative checkpoint - capture your own tenant screenshot after reproducing this result.

Step 10.3 expected outcome (illustrative)

10.4 Compare architectural choices

Native Eventhouse table: choose when real-time KQL performance and ingestion semantics are primary.

Standard shortcut: choose when OneLake remains authoritative and some external-query latency is acceptable.

Accelerated shortcut: choose when you want OneLake ownership with improved KQL query performance and accept cache/capacity considerations.

Expected result: You can state why YellowTaxiNative is native but gold_daily_zone_metrics is

referenced through a shortcut.

Fabric checkpoint - Step 10.4

Expected output

Compare architectural choices

SUCCESS CHECK

You can state why YellowTaxiNative is native but gold_daily_zone_metrics is referenced through a shortcut.

Illustrative checkpoint - capture your own tenant screenshot after reproducing this result.

Step 10.4 expected outcome (illustrative)

Expected outcome visual (illustrative)

Microsoft Fabric - CityRide 360

Stage 10

Workspace

Lakehouse

Data Factory

Warehouse

Real-Time

Monitoring

OneLake shortcut and acceleration

Shortcut

CityRideRT contains a OneLake shortcut to CityRideLake gold_daily_zone_metrics.

Standard query

external_table(...) returns Gold revenue/trip aggregates.

Acceleration test

Query acceleration is enabled and before/after observations are recorded.

Illustrative expected outcome - Fabric UI labels may change

The checkpoint is one Eventhouse containing both a native streaming table and a OneLake shortcut to batch Gold data.

DP-700 concepts practiced: OneLake shortcuts, external_table(), native vs shortcut, query acceleration, data duplication trade-offs

Be able to answer aloud:

- What is the difference between a shortcut and copying data?
- When is accelerated shortcut preferable to native Eventhouse ingestion?

Stage 11 - Security and governance

Exam focus: Practice workspace/item permissions, Warehouse data controls, OneLake security, masking, governance, endorsement, lineage, and audit prerequisites.

Microsoft reference(s): [Secure Fabric data access](#) | [OneLake security](#)

11.1 Document four security personas

Create a small table in your notes: Workspace Admin, Data Engineer, Operations Analyst, External/Restricted Analyst. For each, state the minimum workspace/item/data access required. Do not use broad Workspace Member/Contributor access as a substitute for object-level least privilege.

Expected result: You have a persona-to-access matrix before you configure controls.

Fabric checkpoint - Step 11.1

Expected output

Document four security personas

SUCCESS CHECK

You have a persona-to-access matrix before you configure controls.

Illustrative checkpoint - capture your own tenant screenshot after reproducing this result.

Step 11.1 expected outcome (illustrative)

11.2 Inspect workspace and item permissions

Open Workspace > Manage access and review Admin, Member, Contributor, Viewer. If you have a second identity, assign Viewer temporarily and observe what it can see. Open CityRideDW item permissions and practice granting read access to the item without broad edit access where supported.

Expected result: Your notes distinguish workspace roles from item-level grants.

Fabric checkpoint - Step 11.2

Expected output

Inspect workspace and item permissions

SUCCESS CHECK

Your notes distinguish workspace roles from item-level grants.

Illustrative checkpoint - capture your own tenant screenshot after reproducing this result.

Step 11.2 expected outcome (illustrative)

11.3 Practice Warehouse masking and SQL permissions

```
SQL
CREATE TABLE dbo.DriverSecure (
  DriverId int, FullName varchar(100),
  Email varchar(200) MASKED WITH (FUNCTION = 'email()'),
  LicenseNumber varchar(50) MASKED WITH (FUNCTION = 'partial(0,"XXXX",4)')
);
```

Follow the secure-warehouse lab pattern to practice GRANT/DENY/REVOKE, column-level access, and an RLS security predicate on a small exercise table.

If you do not have a second identity, still create the objects and explain how you would validate them.

Expected result: The Warehouse contains at least one masked table/security exercise and you can explain privileged versus masked views.

Fabric checkpoint - Step 11.3

Expected output

Practice Warehouse masking and SQL permissions

SUCCESS CHECK

The Warehouse contains at least one masked table/security exercise and you can explain privileged versus masked views.

Illustrative checkpoint - capture your own tenant screenshot after reproducing this result.

Step 11.3 expected outcome (illustrative)

11.4 Practice OneLake security

Open CityRideLake > Manage OneLake data access/security. If roles are available to you, create a role that can read Gold tables while excluding Files/source, Files/landing, and Files/quarantine.

If the control is not available due to tenant policy, capture the settings page or documentation and mark the task admin-dependent.

Expected result: You either have a tested OneLake role or a documented admin-dependent limitation with the intended rule.

Fabric checkpoint - Step 11.4

Expected output

Practice OneLake security

SUCCESS CHECK

You either have a tested OneLake role or a documented admin-dependent limitation with the intended rule.

Illustrative checkpoint - capture your own tenant screenshot after reproducing this result.

Step 11.4 expected outcome (illustrative)

11.5 Governance, sensitivity, endorsement, lineage, and audit

Apply a sensitivity label if your tenant has labels configured. Do not fabricate a label if the organization has none.

Promote/endorse the Gold analytical item if you have permission. Open Lineage view and identify source -> transformation -> serving relationships.

If you have Purview Audit permissions, locate Fabric/Power BI activity records. Otherwise document that audit is an organizational/admin surface.

Expected result: You have evidence for every available governance control and a clear note for controls blocked by organizational permissions.

Fabric checkpoint - Step 11.5

Expected output

Governance, sensitivity, endorsement, lineage, and audit

SUCCESS CHECK

You have evidence for every available governance control and a clear note for controls blocked by organizational permissions.

Illustrative checkpoint - capture your own tenant screenshot after reproducing this result.

Step 11.5 expected outcome (illustrative)

Expected outcome visual (illustrative)

Microsoft Fabric - CityRide 360Stage 11

Workspace

Lakehouse

Data Factory

Warehouse

Real-Time

Monitoring

Security and governance

Workspace & item roles

Admin/Contributor/Viewer behavior has been inspected with least-privilege notes.

Data controls

Masking, RLS/CLS/object permissions and OneLake security are configured or documented if admin-dependent.

Governance

Sensitivity label, endorsement, lineage and audit prerequisites are recorded.

Illustrative expected outcome - Fabric UI labels may change

The security checkpoint is not one screen; it is a layered evidence set showing workspace, item, SQL, OneLake, and governance controls.

DP-700 concepts practiced: workspace roles, item permissions, RLS, CLS, object permissions, dynamic data masking, OneLake security, sensitivity labels, endorsement, audit

Be able to answer aloud:

- Why are workspace roles, SQL permissions, OneLake security, and semantic-model RLS not interchangeable?
- Which controls are usually tenant/admin dependent?

Stage 12 - Git, database projects, and deployment pipelines

Exam focus: Use source control and environment promotion; distinguish Git synchronization from deployment pipelines; understand schema-as-code for Warehouse.

Microsoft reference(s): [Git integration](#) | [Deployment pipelines](#)

12.1 Connect DEV to Git

In Workspace settings > Git integration, connect your GitHub or Azure DevOps repository according to what your tenant supports.

Choose a branch (main or a Fabric-specific branch) and initialize/synchronize supported item definitions. Review which CityRide items are source-controllable and which connections/secrets are not stored as plain item logic.

Expected result: The workspace Git status shows connected/synchronized rather than uncommitted for supported items.

Fabric checkpoint - Step 12.1

Expected output

Connect DEV to Git

SUCCESS CHECK

The workspace Git status shows connected/synchronized rather than uncommitted for supported items.

Illustrative checkpoint - capture your own tenant screenshot after reproducing this result.

Step 12.1 expected outcome (illustrative)

12.2 Create and merge a feature branch

Create feature/add-tip-metric in the repository.

Add the view below to CityRideDW, commit the Warehouse definition/schema change, and merge it to main after validation.

```
SQL
CREATE OR ALTER VIEW dbo.vw_ZoneTipMetrics AS
SELECT PickupZoneKey, COUNT(*) AS TripCount,
       AVG(CASE WHEN FareAmount > 0 THEN TipAmount / FareAmount END) AS AvgTipRate
FROM dbo.FactTrip
GROUP BY PickupZoneKey;
```

Synchronize the Fabric workspace with the merged branch and confirm the view is present.

Expected result: Git history shows the feature branch/merge and the DEV workspace contains vw_ZoneTipMetrics.

Fabric checkpoint - Step 12.2

Expected output

Create and merge a feature branch

SUCCESS CHECK

Git history shows the feature branch/merge and the DEV workspace contains vw_ZoneTipMetrics.

Illustrative checkpoint - capture your own tenant screenshot after reproducing this result.

Step 12.2 expected outcome (illustrative)

12.3 Inspect Warehouse database-project representation

In the repo, inspect the Warehouse schema representation. If you use VS Code/SQL Database Projects tooling, make one small schema-as-code change, validate/build it, commit, and synchronize it back.

The learning goal is that database schema can be source-controlled and deployed rather than edited only in a live Warehouse.

Expected result: You can identify where a table/view definition lives in source control.

Fabric checkpoint - Step 12.3

Expected output

Inspect Warehouse database-project representation

SUCCESS CHECK

You can identify where a table/view definition lives in source control.

Illustrative checkpoint - capture your own tenant screenshot after reproducing this result.

Step 12.3 expected outcome (illustrative)

12.4 Create DEV -> TEST -> PROD deployment pipeline

Create DP700-CityRide-TEST and DP700-CityRide-PROD workspaces on Fabric capacity.

Create a Fabric deployment pipeline with DEV, TEST, PROD stages and assign the workspaces.

Deploy supported items DEV -> TEST. Inspect compare/differences. Make one small DEV change, redeploy to TEST, validate, then promote to PROD.

Record which connection/parameter values must be environment-specific rather than hard-coded.

Expected result: The deployment pipeline shows assigned workspaces and at least one successful promotion from DEV to TEST.

Fabric checkpoint - Step 12.4

Expected output

Create DEV -> TEST -> PROD deployment pipeline

SUCCESS CHECK

The deployment pipeline shows assigned workspaces and at least one successful promotion from DEV to TEST.

Illustrative checkpoint - capture your own tenant screenshot after reproducing this result.

Step 12.4 expected outcome (illustrative)

Expected outcome visual (illustrative)

Microsoft Fabric - CityRide 360

Stage 12

Workspace

Lakehouse

Data Factory

Warehouse

Real-Time

Monitoring

Git and deployment lifecycle

Git integration

DEV workspace is connected to GitHub/Azure DevOps and supported item definitions are committed.

Feature workflow

feature/add-tip-metric is merged to main and workspace is synchronized.

Deployment pipeline

DEV -> TEST -> PROD stages show successful deployment / compare status.

Illustrative expected outcome - Fabric UI labels may change

The lifecycle checkpoint is a clean Git status plus a deployment pipeline with DEV/TEST/PROD, not just a public GitHub repository.

DP-700 concepts practiced: Git integration, branching, database projects, deployment pipelines, environment-specific configuration

Be able to answer aloud:

- Why is Git integration not the same as a deployment pipeline?
- What should never be hard-coded or committed as a secret?

Stage 13 - Monitoring, alerts, and systematic break/fix

Exam focus: Use Monitoring hub and engine-specific diagnostics; configure an alert; intentionally fail each major component and document root cause/fix.

Microsoft reference(s): [Monitoring hub lab](#) | [Fabric Monitoring hub](#)

13.1 Inspect Pipeline, Dataflow, and Notebook runs

Open Monitoring hub and filter by DP700-CityRide-DEV. Locate a successful and failed pl_cityride_ingest run.

Open the Copy activity details and record duration, input/output, file/row counts if exposed, and the evaluated error for batch_id=999.

Open a Dataflow refresh and a notebook/Spark run. For the notebook, inspect application/job/stage information and logs.

Expected result: You can navigate from a high-level run to the specific failing activity/job evidence.

Fabric checkpoint - Step 13.1

Expected output

Inspect Pipeline, Dataflow, and Notebook runs

SUCCESS CHECK

You can navigate from a high-level run to the specific failing activity/job evidence.

Illustrative checkpoint - capture your own tenant screenshot after reproducing this result.

Step 13.1 expected outcome (illustrative)

13.2 Create a minimal semantic model and monitor refresh

Create a simple semantic model from CityRideDW or Gold tables using DimDate, DimZone, and FactTrip/gold metrics. Keep it intentionally small; the goal is refresh monitoring, not advanced Power BI modeling.

Trigger or inspect a refresh and record where refresh history appears in your tenant.

Expected result: CityRideModel exists and you have at least one refresh/history entry to discuss.

Fabric checkpoint - Step 13.2

Expected output

Create a minimal semantic model and monitor refresh

SUCCESS CHECK

CityRideModel exists and you have at least one refresh/history entry to discuss.

Illustrative checkpoint - capture your own tenant screenshot after reproducing this result.

Step 13.2 expected outcome (illustrative)

13.3 Inspect Warehouse and Real-Time diagnostics

SQL

```
SELECT * FROM sys.dm_exec_connections;  
SELECT * FROM sys.dm_exec_sessions;  
SELECT * FROM sys.dm_exec_requests;  
  
SELECT TOP 50 *  
FROM queryinsights.exec_requests_history  
ORDER BY start_time DESC;
```

In Real-Time Intelligence, inspect Eventstream live view/data insights and Eventhouse ingestion/query diagnostics.

Expected result: You know a SQL diagnostic surface and a real-time diagnostic surface, not only Monitoring hub.

Fabric checkpoint - Step 13.3

Expected output

Inspect Warehouse and Real-Time diagnostics

SUCCESS CHECK

You know a SQL diagnostic surface and a real-time diagnostic surface, not only Monitoring hub.

Illustrative checkpoint - capture your own tenant screenshot after reproducing this result.

Step 13.3 expected outcome (illustrative)

13.4 Create one real-time alert

Use Fabric Activator/alerting from a real-time query/eventstream if available. Example: alert when 1-minute AvgFare exceeds a threshold or TripCount falls below a demo threshold.

Choose a threshold that is likely to trigger during your test, verify the rule is enabled, then disable it after evidence is captured if desired.

Expected result: An alert rule exists and you know which real-time signal it evaluates.

Fabric checkpoint - Step 13.4

Expected output

Create one real-time alert

SUCCESS CHECK

An alert rule exists and you know which real-time signal it evaluates.

Illustrative checkpoint - capture your own tenant screenshot after reproducing this result.

Step 13.4 expected outcome (illustrative)

13.5 Complete the break/fix matrix

Do every row at least once. Save one incident note per component.

Expected result: You have a reproducible incident note for each component below.

Fabric checkpoint - Step 13.5

Expected output

Complete the break/fix matrix

SUCCESS CHECK

You have a reproducible incident note for each component below.

Illustrative checkpoint - capture your own tenant screenshot after reproducing this result.

Step 13.5 expected outcome (illustrative)

Component	Introduce failure	Diagnose	Fix
-----------	-------------------	----------	-----

Pipeline	p_batch_id=999	Copy activity error, evaluated dynamic path, source/destination settings	Restore valid batch and rerun
Dataflow Gen2	Force invalid LocationID conversion	Refresh history / Applied Step error	Fix type handling and refresh
Notebook	Reference missing column	Notebook output + Spark logs	Correct schema/column and rerun
Eventstream	Break destination mapping / disconnect node	Live view, destination status, data insights	Repair mapping/connection and republish
Eventhouse	Invalid cast/query or ingestion mapping	KQL error / ingestion monitoring	Fix cast/schema/mapping
T-SQL	Misspell table/column or conversion	SQL error + Query Insights/history	Correct query/schema
OneLake shortcut	Point to wrong target or alter access then restore	Shortcut state + external_table error	Restore access/source or recreate shortcut

INCIDENT TEMPLATE

Symptom:
 Affected item/run:
 First diagnostic surface checked:
 Error message / evidence:
 Root cause:
 Fix:
 How I would prevent recurrence:
 DP-700 concept tested:

Expected outcome visual (illustrative)

Microsoft Fabric - CityRide 360

Stage 13

Workspace

Lakehouse

Data Factory

Warehouse

Real-Time

Monitoring

Monitoring, alerts and break/fix

Monitoring hub

Successful and failed pipeline, Dataflow and notebook runs are visible.

Real-time monitoring

Eventstream/Eventhouse health plus an alert rule have been inspected.

Incident notes

Each injected failure has symptom, evidence, root cause, fix and prevention recorded.

Illustrative expected outcome - Fabric UI labels may change

The monitoring checkpoint is evidence-based: successful and failed runs, engine diagnostics, one alert, and completed incident notes.

DP-700 concepts practiced: Monitoring hub, activity output, Spark diagnostics, Warehouse DMVs/Query Insights, Eventstream/Eventhouse monitoring, semantic model refresh, alerts, root-

CityRide 360 | DP-700 Expanded Practical Edition | September 2026

cause analysis

Be able to answer aloud:

- Where do you diagnose a Copy failure versus Spark failure versus KQL ingestion failure?
- What evidence should be captured before changing code or compute?

Stage 14 - Performance optimization lab

Exam focus: Measure before and after; optimize Delta file layout, Spark joins, Warehouse queries, pipelines, and Real-Time workloads based on evidence.

Microsoft reference(s): [Delta optimization](#) | [Warehouse performance](#)

14.1 Capture a baseline before changing anything

Choose one representative Lakehouse query, one Spark join, one Warehouse aggregation, one pipeline run, and one KQL query.

For each, record a timestamp, row volume, duration, and relevant evidence: Delta file count, Spark physical plan, Query Insights entry, pipeline activity duration, or KQL query diagnostics.

Expected result: You have a baseline table before any tuning changes.

Fabric checkpoint - Step 14.1

Expected output

Capture a baseline before changing anything

SUCCESS CHECK

You have a baseline table before any tuning changes.

Illustrative checkpoint - capture your own tenant screenshot after reproducing this result.

Step 14.1 expected outcome (illustrative)

14.2 Optimize the Silver Delta table

```
SPARK SQL
OPTIMIZE dbo.silver_trips;

-- If pickup_date is a frequent filter and your table is large enough to make the test
meaningful:
OPTIMIZE dbo.silver_trips ZORDER BY (pickup_date) VORDER;
```

Record file count/query behavior before and after. Do not claim a speed-up if the dataset is too small to show one.

Expected result: You can explain the small-files problem, V-Order, and Z-Order with your own measured evidence.

Fabric checkpoint - Step 14.2

Expected output

Optimize the Silver Delta table

SUCCESS CHECK

You can explain the small-files problem, V-Order, and Z-Order with your own measured evidence.

Illustrative checkpoint - capture your own tenant screenshot after reproducing this result.

Step 14.2 expected outcome (illustrative)

14.3 Compare regular join and broadcast join

PYTHON

```
from pyspark.sql.functions import broadcast

trips = spark.table("dbo.silver_trips")
zones = spark.table("dbo.silver_zones")

regular = trips.join(zones, trips.PULocationID == zones.LocationID, "left")
regular.explain("formatted")

fast_join = trips.join(broadcast(zones), trips.PULocationID == zones.LocationID, "left")
fast_join.explain("formatted")
```

Compare the plans and note whether the broadcast removes a shuffle. Broadcast only makes sense because the zone dimension is small.

Expected result: Your notes contain two formatted plans and a short explanation of the difference.

Fabric checkpoint - Step 14.3

Expected output

Compare regular join and broadcast join

SUCCESS CHECK

Your notes contain two formatted plans and a short explanation of the difference.

Illustrative checkpoint - capture your own tenant screenshot after reproducing this result.

Step 14.3 expected outcome (illustrative)

14.4 Tune a Warehouse query based on Query Insights

```
SQL
SELECT PickupZoneKey, COUNT(*) AS Trips, SUM(TotalAmount) AS Revenue
FROM dbo.FactTrip
WHERE PickupDateKey BETWEEN 20200101 AND 20991231
GROUP BY PickupZoneKey;

SELECT * FROM queryinsights.long_running_queries;
```

Then narrow filters to the actual date range, select only required columns, and avoid implicit type conversions. Understand that statistics are important to the optimizer; use CREATE/UPDATE STATISTICS only when justified by the workload and supported behavior.

Expected result: You have Query Insights evidence and a before/after query explanation.

Fabric checkpoint - Step 14.4

Expected output

Tune a Warehouse query based on Query Insights

SUCCESS CHECK

You have Query Insights evidence and a before/after query explanation.

Illustrative checkpoint - capture your own tenant screenshot after reproducing this result.

Step 14.4 expected outcome (illustrative)

14.5 Simplify orchestration and real-time queries

Pipeline: remove redundant copies, parallelize independent activities only where dependencies allow, and reuse parameters rather than cloning pipelines.

KQL: filter by time early, project only needed columns, choose sensible bin sizes, and compare native Eventhouse data with shortcut-based Gold data.

Eventstream: push simple filtering/aggregation upstream when it reduces downstream volume without hiding data you still need.

Expected result: Your optimization log lists what changed, why, and what evidence improved or did not improve.

Fabric checkpoint - Step 14.5

Expected output

Simplify orchestration and real-time queries

SUCCESS CHECK

Your optimization log lists what changed, why, and what evidence improved or did not improve.

Illustrative checkpoint - capture your own tenant screenshot after reproducing this result.

Step 14.5 expected outcome (illustrative)

Workload	Baseline evidence	Change	Observed result
Delta		OPTIMIZE / ZORDER / VORDER	
Spark		Broadcast small zone dimension	
Warehouse		Filter/project/type cleanup	
Pipeline		Remove redundant activities	
Eventhouse/KQL		Time filter / projection / shortcut comparison	

Expected outcome visual (illustrative)

Microsoft Fabric - CityRide 360

Stage 14

Workspace

Lakehouse

Data Factory

Warehouse

Real-Time

Monitoring

Performance optimization

Baseline

Query duration, file count, Spark plan and activity duration captured before tuning.

Changes

OPTIMIZE/ZORDER/VORDER, broadcast join, SQL filtering/statistics and pipeline simplification tested.

Evidence

Before/after table documents observed differences without claiming unrealistic gains.

Illustrative expected outcome - Fabric UI labels may change

The optimization checkpoint is a measurement log. DP-700 expects evidence-based tuning, not a list of commands.

DP-700 concepts practiced: Delta optimization, V-Order, Z-Order, Spark physical plan, broadcast join, Warehouse Query Insights, pipeline optimization, KQL optimization

Be able to answer aloud:

- Why measure before changing Spark compute?
- When can an optimization command add cost without meaningful benefit?

Stage 15 - Final exam simulation and project sign-off

Exam focus: Turn implementation into scenario reasoning; rebuild core patterns without a recipe; prove coverage with evidence.

Microsoft reference(s): [Official DP-700 study guide](#) | [DP-700 interactive exercises](#)

15.1 30-minute rebuild challenge - updated for the Fabric-native source

Start from a blank practice workspace or write the sequence from memory if capacity is limited.

- 1) Create a Lakehouse and seed NYC Taxi - Green from Fabric Sample data into a source table.
- 2) Create at least one Files/source/trips/batch_id=... Parquet batch and a parameterized Lakehouse Files -> landing Copy activity.
- 3) Add a notebook parameter cell, read the landed Parquet, clean invalid rows, aggregate by date/location, and write a Delta Gold table.
- 4) Create a Warehouse table and run one T-SQL aggregation/window query.
- 5) Create an Eventstream with Yellow Taxi sample, route to Eventhouse, and run one KQL one-minute aggregation.
- 6) Open Monitoring hub and locate the new runs.

Expected result: You can reproduce the architecture without external HTTP or an Azure credential, and you know where to troubleshoot each engine.

Fabric checkpoint - Step 15.1

Expected output

30-minute rebuild challenge - updated for the Fabric-native source

SUCCESS CHECK

You can reproduce the architecture without external HTTP or an Azure credential, and you know where to troubleshoot each engine.

Illustrative checkpoint - capture your own tenant screenshot after reproducing this result.

Step 15.1 expected outcome (illustrative)

15.2 Answer the scenario questions aloud

Why use Lakehouse rather than Warehouse for the Bronze/Silver Spark path?

When would you use Warehouse instead of Lakehouse SQL analytics endpoint?

Why Dataflow Gen2 for zones but PySpark for trips?

When is a pipeline incremental load better than mirroring?

What is the difference between mirroring and a OneLake shortcut?

Why ingest into Eventhouse natively instead of querying a OneLake shortcut?

When would an accelerated shortcut be preferable?

What problem does a Spark watermark solve?

Where do you diagnose Copy vs Spark vs KQL failures?

How do workspace roles, OneLake security, SQL RLS/CLS, and semantic-model RLS differ?

What evidence would you collect before changing Spark compute?

Why is Git integration not the same as a deployment pipeline?

Expected result: You can answer each question in 30-90 seconds using evidence from your own project.

Fabric checkpoint - Step 15.2

Expected output

Answer the scenario questions aloud

SUCCESS CHECK
You can answer each question in 30-90 seconds using evidence from your own project.

Illustrative checkpoint - capture your own tenant screenshot after reproducing this result.

Step 15.2 expected outcome (illustrative)

15.3 Complete the final evidence checklist

Do not mark an item done only because you read about it. Keep one of: screenshot, run ID, code/query result, commit, or incident note as evidence.

Expected result: The checklist below is complete and every checked item has evidence.

Fabric checkpoint - Step 15.3

Expected output

Complete the final evidence checklist

SUCCESS CHECK

The checklist below is complete and every checked item has evidence.

Illustrative checkpoint - capture your own tenant screenshot after reproducing this result.

Step 15.3 expected outcome (illustrative)

Deliverable	Done
Workspace/Spark/domain/OneLake/Airflow settings inspected	☐
Git + database project + deployment pipeline practiced	☐
Workspace/item/SQL/OneLake security practiced	☐
Masking, sensitivity/endorsement/audit concepts covered	☐
Pipeline parameters, expressions, schedule/event triggers practiced	☐
Full + incremental file-based batch load completed	☐
Dataflow Gen2 reference transform completed	☐
PySpark Bronze/Silver/Gold completed	☐
Duplicate/missing/invalid data handling completed	☐
Warehouse star schema + T-SQL load completed	☐
SQL window/aggregation queries completed	☐
Fabric SQL database mirroring verified	☐
Eventstream Yellow Taxi -> native Eventhouse completed	☐
KQL filtering/aggregation/windowing completed	☐
Spark Structured Streaming + watermark/window completed	☐
OneLake Eventhouse shortcut completed	☐
Standard vs accelerated shortcut compared	☐
Monitoring hub + Warehouse monitoring completed	☐
Semantic model refresh monitoring practiced	☐
Alert configured	☐
All break/fix scenarios completed	☐
Lakehouse/Spark/Warehouse/pipeline/RTI optimization experiments completed	☐

Expected outcome visual (illustrative)

Workspace

Lakehouse

Data Factory

Warehouse

Real-Time

Monitoring

Final exam simulation

Rebuild challenge

From blank workspace: sample source -> parameterized OneLake batch copy -> notebook -> Warehouse -> Eventstream/KQL.

Scenario reasoning

You can answer store/engine/security/incremental/monitoring trade-off questions aloud.

Sign-off

All checklist items have evidence: screenshots, run IDs, queries, code and notes.

Illustrative expected outcome - Fabric UI labels may change

The final checkpoint is a complete evidence pack: real screenshots from your tenant, code/query outputs, run history, incidents, Git commits, and deployment status.

DP-700 concepts practiced: end-to-end scenario reasoning, rebuild under time pressure, tool selection, troubleshooting, evidence-based optimization

Be able to answer aloud:

- Can you explain not only what you built, but why each Fabric item was chosen?
- Can you rebuild a working subset in 30 minutes without copying a tutorial?

Suggested 3-day execution plan

Day	Build	What to capture
Day 1 - Batch foundation	Stages 1-6: workspace, seed sample, source batches, parameterized pipeline, Dataflow, PySpark, Warehouse	Workspace item list; seed table; source/landing folders; pipeline run; Dataflow steps; Bronze/Silver/Gold; star schema
Day 2 - Real time + security + ALM	Stages 7-12: SQL DB/mirroring, Eventstream/Eventhouse, Structured Streaming, shortcuts, security, Git/deployment	Mirrored table; Eventstream canvas; KQL result; stream table; shortcut query; security evidence; Git/deployment status
Day 3 - Operations + rehearsal	Stages 13-15: monitoring, alert, break/fix, optimization, exam simulation	Failed/success runs; alert; incident notes; before/after metrics; final checklist

Official references and exact labs

- [Official DP-700 study guide](https://learn.microsoft.com/en-us/credentials/certifications/resources/study-guides/dp-700) - <https://learn.microsoft.com/en-us/credentials/certifications/resources/study-guides/dp-700>
- [Microsoft Fabric interactive exercises \(filter DP-700\)](https://microsoftlearning.github.io/mslearn-fabric/) - <https://microsoftlearning.github.io/mslearn-fabric/>
- [Data ingestion options for a Lakehouse](https://learn.microsoft.com/en-us/fabric/data-engineering/load-data-lakehouse) - <https://learn.microsoft.com/en-us/fabric/data-engineering/load-data-lakehouse>
- [Data Factory tutorial - NYC Taxi sample](https://learn.microsoft.com/en-us/fabric/data-factory/tutorial-end-to-end-pipeline) - <https://learn.microsoft.com/en-us/fabric/data-factory/tutorial-end-to-end-pipeline>
- [Load sample data with a Pipeline Copy activity](https://learn.microsoft.com/en-us/fabric/data-factory/tutorial-load-sample-data-to-data-warehouse) - <https://learn.microsoft.com/en-us/fabric/data-factory/tutorial-load-sample-data-to-data-warehouse>
- [Fabric Eventstreams overview](https://learn.microsoft.com/en-us/fabric/real-time-intelligence/event-streams/overview) - <https://learn.microsoft.com/en-us/fabric/real-time-intelligence/event-streams/overview>
- [Eventstream data preview](https://learn.microsoft.com/en-us/fabric/real-time-intelligence/event-streams/preview-data) - <https://learn.microsoft.com/en-us/fabric/real-time-intelligence/event-streams/preview-data>
- [OneLake shortcuts](https://learn.microsoft.com/en-us/fabric/onelake/onelake-shortcuts) - <https://learn.microsoft.com/en-us/fabric/onelake/onelake-shortcuts>
- [Query acceleration](https://learn.microsoft.com/en-us/fabric/real-time-intelligence/query-acceleration-overview) - <https://learn.microsoft.com/en-us/fabric/real-time-intelligence/query-acceleration-overview>
- [Git integration](https://learn.microsoft.com/en-us/fabric/cicd/git-integration/intro-to-git-integration) - <https://learn.microsoft.com/en-us/fabric/cicd/git-integration/intro-to-git-integration>
- [Deployment pipelines](https://learn.microsoft.com/en-us/fabric/cicd/deployment-pipelines/intro-to-deployment-pipelines) - <https://learn.microsoft.com/en-us/fabric/cicd/deployment-pipelines/intro-to-deployment-pipelines>
- [Delta optimization and V-Order](https://learn.microsoft.com/en-us/fabric/data-engineering/delta-optimization-and-v-order) - <https://learn.microsoft.com/en-us/fabric/data-engineering/delta-optimization-and-v-order>
- [Warehouse performance guidance](https://learn.microsoft.com/en-us/fabric/data-warehouse/guidelines-warehouse-performance) - <https://learn.microsoft.com/en-us/fabric/data-warehouse/guidelines-warehouse-performance>

Appendix A - Command quick reference

PYSPARK

```
df.select(...)
df.filter(...) / df.where(...)
df.withColumn(...)
df.dropDuplicates([...])
df.fillna({...})
df.join(other, keys, "left")
df.groupBy(...).agg(...)
broadcast(small_df)
df.write.format("delta").mode(...).saveAsTable(...)
df.writeStream.withWatermark(...).groupBy(window(...)).writeStream...
```

KQL

Table

```
| where ...
| project ...
| extend ...
| summarize Count=count(), Revenue=sum(...) by bin(Timestamp, 5m)
| order by ... desc
```

```
external_table('ShortcutName')
| summarize ...
```

SQL

```
SELECT / JOIN / GROUP BY / HAVING
WITH ... AS (...)
INSERT ... SELECT ...
CREATE OR ALTER VIEW ...
SUM(...) OVER(PARTITION BY ... ORDER BY ...)
GRANT / DENY / REVOKE
Dynamic data masking
RLS security predicates
DMVs + queryinsights views
```

Appendix B - DP-700 decision cheat sheet

Question	Default mental model
Pipeline vs Dataflow Gen2 vs Notebook?	Pipeline orchestrates/moves; Dataflow is low-code Power Query shaping; Notebook is code-first Spark/custom processing.
Lakehouse vs Warehouse?	Lakehouse for open Delta/Spark engineering; Warehouse for relational T-SQL dimensional serving.
Eventhouse vs Lakehouse for streams?	Eventhouse for low-latency event/time-series KQL; Lakehouse for durable open Delta and broad Spark/SQL reuse.
Mirroring vs Copy Data?	Mirroring continuously replicates supported source changes; Copy is explicit movement you orchestrate.
Shortcut vs copy?	Shortcut references data without copying; copy creates a managed duplicate plus ingestion process.
Full vs incremental?	Full reprocesses all; incremental moves only new/changed data using files, watermark, CDC, or streams.
Spark watermark?	Bounds state for late event-time data and trades correctness horizon against state/resource usage.
Optimize or measure first?	Measure first with monitoring, query history, Spark plans, file layout, and activity metrics.